Expert Systems With Applications 215 (2023) 119359

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Hybrid genetic algorithm with variable neighborhood search for flexible job shop scheduling problem in a machining system☆,☆☆

Kexin Sun [a,b], Debin Zheng [c], Haohao Song [c], Zhiwen Cheng [d,*], Xudong Lang [a,b], Weidong Yuan [a,b], Jiquan Wang [c,*]

[a] AVIC Nanjing Engineering Institute of Aircraft Systems, Nanjing 211106, China
[b] Aviation Key Laboratory of Science and Technology on Aero Electromechanical System Integration, Nanjing 211106, China
[c] College of Engineering, Northeast Agricultural University, Harbin 150030, China
[d] College of Management and Economics, Tianjin University, Tianjin 300072, China

ARTICLE INFO

Keywords:
Flexible job shop scheduling problem
Variable neighborhood search
Hybrid genetic algorithm
Combined crossover operator
Combined mutation operator

ABSTRACT

This paper proposes an improved hybrid genetic algorithm with variable neighborhood search (HGA-VNS) for addressing the flexible job shop scheduling problem (FJSP) considering the machine work load balance in machining system, with the minimization of the makespan. In the HGA-VNS algorithm, each solution is represented by a chromosome consists of two parts, where the first part is the code of the machining machine number, and the second part chromosome is the code of the machining process number. Second, considering the slow convergence speed of the previous algorithms, a combined methods in crossover and mutation operators that considers the machine work load balance is proposed. Third, a local search approach that carry out for key processes on the critical path which reduces the number of invalid transformations is proposed. For the HGA-VNS, using the orthogonal experiment approach, the best combination of parameters is provided. Then, the proposed HGA-VNS is tested on sets of extended instances based on the well-known benchmarks. Experimental results show that the HGA-VNS is effective, and its performance is significantly better than other algorithms in solving flexible job shop problems in a machining system. Finally, the proposed HGA-VNS is applied to optimize practical FJSP in enterprise F. Compared with the original scheduling scheme, the makespan of the optimal scheduling scheme is reduced by 14.92%, and HGA-VNS can obtain more efficient and economic solutions.

1. Introduction

Production scheduling plays a vital role in many manufacturing systems. An effective production scheduling program can improve the production efficiency and resource utilization of an enterprise (Zhang & Ming, 2020; Pezzella, Morganti, & Ciaschetti, 2008). There are many types of production scheduling problems, and job shop scheduling (JSP) problem is one of the most difficult types to solve. The JSP problem is the most famous basic scheduling problem, and it is also a NP-hard problem (Garey et al., 1976). Brucker and Schlie (1990) extended the JSP problem in 1990 and proposed the concept of flexible job shop scheduling problem (FJSP) for the first time. In the FJSP, the different operations of each job can be processed on different machines with different processing times, and the staff can choose the machine according to the actual situation (Pan, Lei, & Wang, 2022; Ding & Gu, 2020a; Gu, Huang, & Liang, 2019). FJSP is closer to actual production than JSP problem, which also makes it more difficult to solve.

The FJSP is currently-one of the most critical issues in process planning and manufacturing, and it is also one of the most effective methods for solving multiple varieties and small batch production problems in discrete manufacturing enterprises. The FJSP is studied with the goal of achieving the shortest makespan. Recently, many scholars have conducted effective research on solving FJSP, and proposed or improved many new intelligent optimization methods. Wang, Li, and Li (2021) proposed a multi-population collaborative genetic algorithm (MPCGA) based on collaborative optimization. With the optimization goal of minimizing the maximum makespan, the MPCGA independently optimizes two sub-problems. Compared with other

☆ Kexin Sun, Debin Zheng, and Haohao Song, **these authors contributed equally to this work.** ☆☆ This work is supported by National Social Science Foundation (Grant No. 21BGL174).

* Corresponding authors.
E-mail addresses: zhiwenchang2017@163.com (Z. Cheng), wang-jiquan@163.com (J. Wang).

https://doi.org/10.1016/j.eswa.2022.119359
Received 25 August 2022; Received in revised form 13 October 2022; Accepted 24 November 2022
Available online 29 November 2022
0957-4174/© 2022 Elsevier Ltd. All rights reserved.

algorithms, the MPCGA has achieved better results. Yan et al. (2021) addressed to the constraint influence imposed by finite transportation conditions in the FJSP, and designed a three-layer encoding with redundancy and decoding with correction to improve the genetic algorithm and solve FJSP model under finite transportation conditions. The results of the experiment show that the finite transportation conditions have a significant impact on FJSP under different scales of scheduling problems and transportation times. Fan, Zhang, Liu, Shen, and Gao (2022) investigated a flexible job shop scheduling problem with machine reconfigurations (FJSP-MR) to minimize the total weighted tardiness (TWT). Numerical experimental results indicate that the proposed improved genetic algorithm is highly efficient for the FJSP-MR in a variety of scenarios, where the specially designed local search is an effective complementary component for the basic genetic algorithm. Pan et al. (2022) considered to handle uncertainty and proposed a bi-population evolutionary algorithm with feedback to minimize fuzzy makespan and fuzzy total energy consumption and maximize minimum agreement index. Extensive experiments are conducted to test the performance of evolutionary algorithm with feedback, and the obtained results show that evolutionary algorithm with feedback can provide promising results. In addition, simulated-annealing-based hyper-heuristic (Lim, Wong, & Chin, 2022), multi-Agent reinforcement learning (RL) (Johnson, Chen, & Lu, 2022), hybrid of human learning optimization (HLO) algorithm and particle swarm optimization algorithm (PSO) with scheduling strategies (Ding & Gu, 2020a), improved PSO algorithm based novel encoding and decoding schemes (Ding & Gu, 2020b), distributed particle swarm algorithm (Nouiri et al., 2018), hybrid harmony search (HHS) algorithm based on integrated method (Yuan, Xu, & Yang, 2013), and parallel variable neighborhood search (PVNS) algorithm (Yazdani et al., 2010)were also used to solve FJSP.

Although scholars have proposed many methods that can solve FJSP problems, there is no algorithm that is superior to other algorithms in solving any FJSP (Qais et al., 2018). Therefore, it is necessary to choose an intelligent optimization method with better performance for solving FJSP. Because genetic algorithm (GA) has the characteristics of good parallelism, adaptability and autonomous learning, it has attracted the attention of scholars from all over the world. The GA has a strong global search capability. It can quickly find a large number of solutions in the search space when solving simple problems, but it has limitations when solving complex problems. Therefore, many literatures focus on hybrid genetic algorithms (HGA) (Fekih et al., 2021; Wang & Zhu, 2021; Jamrus, Chien, Gen & Sethanan, 2018; Qin, Cheng, Zhang, Li, & Shi, 2016; Chang, Chen, Liu, & Chou, 2015; Al-Hinai & ElMekkawy, 2011; Zhang, Rao, & Li, 2008; Gao, Gen, Sun, & Zhao, 2007). Since the HGA takes into account the performance advantages of GA and local search algorithm, HGA is undoubtedly a better choice for solving FJSPs (Cheng, Gen, & Tsujimura, 1999).

In the research of HGA, Gao et al. (2007) proposed a genetic algorithm based on the bottleneck transfer strategy for local search. This method adopting new crossover and mutation operators is designed for the new chromosome structure, dynamically adjusting the neighborhood structure during the local search process to enhance the search ability of the algorithm. Finally, a large number of experimental results verify the effectiveness of the algorithm. Tang, Zhang, Lin, and Zhang (2011) proposed a HGA to solve the FJSP, which hybridized the chaotic particle swarm algorithm (PSO) with the GA. The main innovation of the algorithm is to mix the random initialization strategy and Kacem allocation scheme according to a certain ratio to generate a population, which enhances the diversity of the initial population. Experimental results show that compared with other existing initial population generation methods, this method has a faster solution speed and higher solution quality. Qin et al. (2016) hybridized the tabu search (TS) algorithm with the GA, and proposed a HGA with the minimum of the maximum makespan as the objective function to solve the FJSP. This hybrid algorithm takes advantage of the strong global search ability of GA and strong local search ability of TS. Experimental results show that the algorithm has a significant improvement in solution accuracy and calculation speed. Chang and Liu (2017) proposed a HGA to solve the distributed flexible job shop scheduling problem (DFJSP). The Taguchi method is used when designing the parameters of the algorithm. In addition, the proposed algorithm combines multiple crossover and mutation operators into an evolutionary framework, which improves the diversity of the population and accelerates the optimization speed of the algorithm. Experimental results show that the algorithm has strong robustness and fast calculation speed. Jamrus et al. (2018) developed a hybrid approach integrating a particle swarm optimization algorithm with a Cauchy distribution and genetic operators (HPSO + GA) for solving an FJSP by finding a job sequence that minimizes the makespan with uncertain processing time. The results show the practical viability of this approach. Wang and Zhu (2021) first presented a mathematical model with the objective to minimize the makespan, then proposed a hybrid algorithm (HGA-TS) which combines genetic algorithm (GA) and tabu search (TS) to solve the FJSP with sequence-dependent set-up times and job lag times. The results show the great performance of HGA-TS in solving FJSP with sequence-dependent set-up times and job lag times. Fekih, Hadda, Kacem, and Hadj-Alouane (2021) developed a hybrid approach combining genetic algorithms and the Tabu search metaheuristic to addresses the FJSP under partial and total flexibility with the objective of minimizing the total completion time. The experimental results prove the effectiveness and efficiency of the proposed hybridization.

From the recent researches on the precast production system, find that: (1) there are some several literatures considered solving FJSPs, however, fewer of them have considered realistic constraints; (2) the mechanically produced process can reduce production waste levels while considerably improving production efficiency levels, however, efficient algorithms are needed to consider both problem structures and objective features in the mechanical production process. Therefore, in this study, we propose hybrid genetic algorithm with variable neighborhood search (HGA-VNS) as a means of addressing the mechanical production process. In the proposed HGA-VNS algorithm, we hybridize GA and variable neighborhood search (VNS) algorithm (Mladenović & Hansen, 1997) to speed up the convergence of the algorithm. Besides, the combined crossover operator inherits the excellent global search ability of the GA, and the combined mutation operator and the variable neighborhood search operator enhance the local search ability of the algorithm.

The main contributions of the proposed HGA-VNS are as follows:

(1) Each solution is represented by a chromosome consists of two parts, where the first part chromosome is the code of the machining machine number, and the second part chromosome is the code of the machining process number.
(2) Because of the slow convergence speed of the previous algorithms, an efficient HGA-VNS is developed.
(3) Efficient combined methods in crossover and mutation operators that considers the machine work load balance is proposed.
(4) A local search approach that carry out for key processes on the critical path which reduces the number of invalid transformations is proposed.
(5) For the HGA-VNS, using the orthogonal experiment approach, the best combination of parameters is provided.

The work stages of HGA-VNS for FJSP in machining system are shown in Fig. 1, and the specific improvements to HGA-VNS are described in detail below.

The rest of this paper is organized as follows: Section 2 briefly describes the FJSP. Next, Section 3 illustrates the framework of the proposed HGA-VNS, including algorithm encoding, decoding, population initialization, improved genetic operations and variable neighborhood search. In Section 4, simulation experiments and result analysis are carried out, including the investigation of the optimal parameter

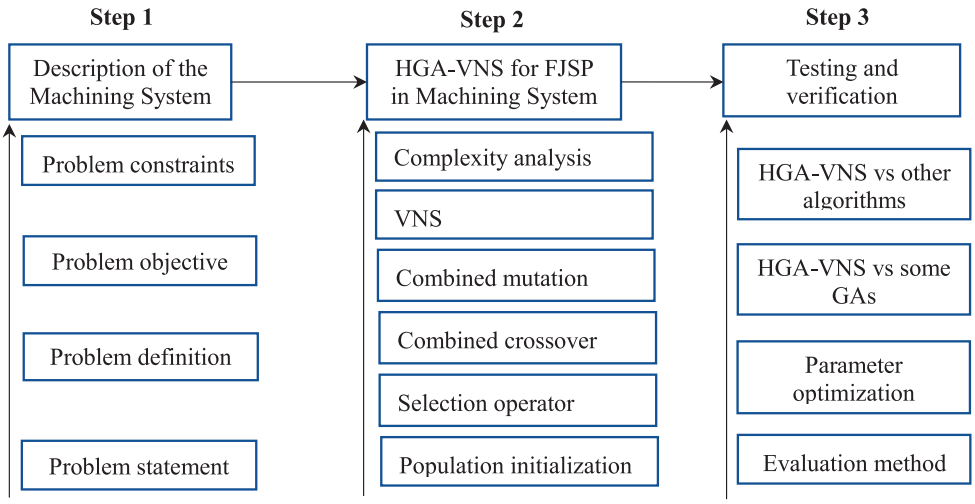


Fig. 1. The work stages of HGA-VNS for FJSP in Machining System.

combination of the algorithm and comparison with the results of other algorithms to solve the calculation examples. In Section 5, the proposed HGA-VNS is applied to optimize practical FJSP in enterprise F. Finally, the last section presents the concluding remarks and future research directions.

2. Problem description of FJSP in a machining system

2.1. Problem statement

Enterprise F has a machine shop that processes a Q-type part for its main product. The shop has n types of jobs, each of which has a different operation, for a total of T_0 operations. The shop has m types of equipment to machine these jobs, with a total of M_0 machining machines. Among them, the machining center can complete turning, milling and drilling, but not grinding, and the other machining machines can only complete one kind of machining tasks.

In the national macro-control of the market economy, the domestic economy began to fully rebound. Driven by the strong demand for construction machinery from the economic recovery, the market demand for Q-type parts also surged. In order to meet the market demand, enterprise F had to expand the production of Q-type parts. However, when expanding the production of this type of part, the scheduling scheme of the existing machining system often leads to load imbalance problems on the machining line. In serious cases, it also leads to blockage of the machining line and even causes the machining line to stop for maintenance, which seriously affects the production schedule of the enterprise and brings huge economic losses to the enterprise. Therefore, how to solve the unbalanced load of the parts processing process in the machining system is a real problem that enterprises F urgently need to solve.

2.2. Problem definition

The scheduling for enterprise F can be defined (Zhang, Gao, & Shi, 2011) as follows: n types of jobs ($J=\{J_1,J_2,J_3,\ldots,J_n\}$) are processed on m types of machines ($M=\{M_1,M_2,M_3,\ldots,M_m\}$). Each job has one or more operations, and the process sequence of operations in each job is predetermined. Each operation in any job can be processed by one machine out of a predetermined set of capable machines. When the operation is carried out on different machines, the processing time of operation is not necessarily the same. FJSP is going to assign each operation to a suitable machine and determine the sequence of assigned operations on each machine to minimize the makespan. Some of the symbols that appear in the FJSP and their corresponding meanings are shown in Table 1.

Table 1
Parameter symbol and its meaning in FJSP.

Symbol	The meaning of the symbol
n	The number of job types
J_i	The job number
m	The number of machine types
M_j	The machine number
O_{ij}	The j-th operation of the i-th job
T_0	The total number of operations of all jobs
M_0	The total number of machines
T_{min}	The minimum makespan
N	The total number of all scheduling schemes
C_j	The completion time of the j-th scheduling scheme
J_M	Machine information matrix
T	Processing time matrix
m_{jh}	The number of optional machines for the h-th operation of job j

2.3. Problem objective function

When solving the FJSP for enterprise F, in order to measure the pros and cons of the scheduling scheme, the minimum makespan is selected as the evaluation index. The makespan refers to the longest time in the finishing times of the last operation of all jobs. The smaller the makespan, the higher the production efficiency of the job shop. Let N be the total number of all scheduling schemes, C_j be the completion time of the j-th scheduling scheme. Then, the minimum makespan can be expressed as

$$T_{\min} = \min\left(\max_{1\leqslant j\leqslant n}(C_j)\right) \tag{1}$$

2.4. Problem constraints

When establishing the mathematical model of the FJSP for Enterprise F, some constraints in actual production need to be met. The constraints (Karimi et al., 2012) are as follows:

(1) All jobs have the same priority.
(2) Only one job can be processed on each machine at each time.
(3) At any given time, a job can only be processed on one machine.
(4) Each operation can be processed without interruption on one of a set of available machines.
(5) All jobs start at time 0 and all machines are available at time 0.
(6) The order of operations for each job is predefined and cannot be modified.

(7) Transition time between machines and machine setup time are not considered.

3. Proposed HGA-VNS for FJSP

3.1. Encoding scheme

When using GAs to solve FJSP, each scheduling scheme must use a specific coding method, which is convenient for using genetic operators to operate to generate new solutions. A good coding scheme can provide a reasonable scale of solution space, reduce the generation of illegal solutions, and improve the search capability of the algorithm.

Compared with integrated coding, segmented coding can perform more crossover and mutation operations. The chromosome using segmented coding is composed of two parts, A and B. Part A is the machine selection (MS) part, and part B is the operation sequence (OS) part, and their lengths are all T_0 (the total number of operations of all jobs).

Take the FJSP in Table 2 as an example. Table 2 is the optional machine and its processing time in the FJSP of three jobs and seven operations. Fig. 2 shows the structure of a chromosome generated by segmented coding for the FJSP in Table 2.

(1) Machine section part.

In Fig. 2, the left half of the chromosome is the MS part, and the numbers indicate the sequence numbers of the processing machines selected by the process in the set of optional processing machines. For example, the first number "3" in the machine selection part corresponds to process O_{11}. The set of optional processing machines for O_{11} is $M=\{M_1, M_2, M_3\}$, and "3" means the third machine in M, which is machine M_3.

(2) Operation sequence part.

In Fig. 2, the right half of the chromosome is the OS part, whose function is to determine the processing sequence of each operation of each job. The first number is "2", which means the first operation of job J_2; the second number is also "2", which means the second operation of job J_2; the third number is "3", which represents the first operation of job J_3. By analogy, the processing sequence of each job is as follows:

$O_{21} \rightarrow O_{22} \rightarrow O_{31} \rightarrow O_{11} \rightarrow O_{12} \rightarrow O_{23} \rightarrow O_{32}$.

Corresponding to the MS part and the OS part, a scheduling plan for determining the processing machine and operation sequence is obtained, as shown in Fig. 3.

3.2. Decoding scheme

The segmented-encoded chromosome consists of two parts, and the MS part and the OS part need to be decoded separately. The key is to decode the OS part into the active schedule corresponding to the MS part.

Table 2
The FJSP of three jobs and seven operations.

Jobs	Operations	Optional machine and its processing time		
		M_1	M_2	M_3
J_1	O_{11}	3	5	8
	O_{12}	7	3	–
J_2	O_{21}	–	8	3
	O_{22}	4	8	–
	O_{23}	–	3	8
J_3	O_{31}	–	3	6
	O_{32}	–	–	5

Note: "-" means that the current machine cannot process the current operation.

Machine Section (MS)							Operation Sequence (OS)						
3	1	1	1	2	2	1	2	2	3	1	1	2	3
O_{11}	O_{12}	O_{21}	O_{22}	O_{23}	O_{31}	O_{32}	J_2	J_2	J_3	J_1	J_1	J_2	J_3

Fig. 2. Schematic diagram of chromosome structure.

O_{21}	O_{22}	O_{31}	O_{11}	O_{12}	O_{23}	O_{32}
M_2	M_1	M_3	M_3	M_1	M_3	M_3

Fig. 3. A scheduling scheme for FJSP.

(1) Machine selection partial decoding.

Decode each gene of the MS part of the chromosome in the order from left to right. When decoding, the information of machines and process time contained in each gene should be converted and stored in matrix J_M and matrix T respectively. The row and column indexes of matrix J_M and matrix T respectively represent the job and the operation of the corresponding job. For example, $J_M(j,h)$ represents the sequence number of the machine selected in the h-th operation of job j in the optional processing machine set; $T(j,h)$ represents the time consumed for processing the h-th operation of job j.

Now we take the chromosome of the FJSP in Table 2 and Fig. 1 as an example to explain the decoding operation. Assuming that the MS part of the chromosome is [3 1 1 1 2 2 1], and its first gene is the number "3", it means that the processing machine of operation O_{11} is M_3, and the corresponding processing time is 8. The decoding of other genes later can be deduced by analogy, and the process is as follows:

$$\begin{bmatrix} O_{11} & O_{12} & O_{13} \\ O_{21} & O_{22} & O_{23} \\ O_{31} & O_{32} & O_{33} \end{bmatrix} \tag{2}$$

$$J_M = \begin{bmatrix} 3 & 1 & \sim \\ 2 & 1 & 3 \\ 3 & 3 & \sim \end{bmatrix} \tag{3}$$

$$T = \begin{bmatrix} 8 & 7 & \sim \\ 8 & 4 & 8 \\ 6 & 5 & \sim \end{bmatrix} \tag{4}$$

Eq. (2) represents the operations at the corresponding position in the matrix, Eq. (3) is the machine sequence matrix, Eq. (4) is the time sequence matrix, and "∼" indicates that there is no operation at this position.

(2) Operation sequence partial decoding.

In the decoding process, a sequential decoding method is used to decode the OS part in the order from left to right. Now we take the chromosome in Fig. 2 as an example. Starting from the operation O_{21}, it can be seen from the matrix J_M that the operation O_{21} is processed on the machine M_2, and then O_{22} is processed on the machine M_1, and so on. It should be pointed out that the constraint conditions in the mathematical model need to be met during the decoding process. Fig. 4 is the scheduling scheme after decoding.

3.3. Population initialization

In order to enhance the diversity of the population, so that the individuals in the population are dispersed in the solution space, a random initialization method is adopted.

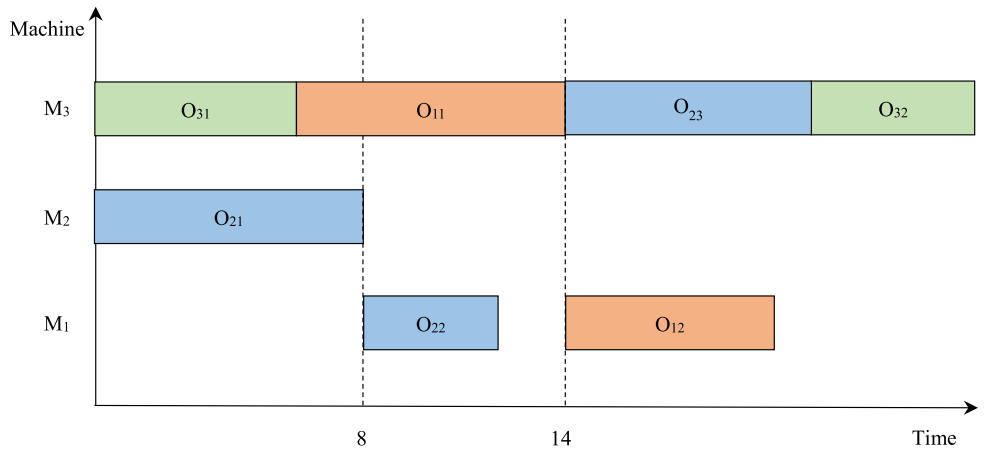


Fig. 4. Gantt chart of the scheduling plan.

(1) Generation of MS part chromosomes.

The steps to generate MS chromosomes are as follows:

Step1: Select the first operation of the first job in the job set;

Step2: Randomly generate an integer R in the interval [1, m_{jh}], where m_{jh} is the number of optional machines for the h-th operation of job j, and set this random number to the value of the current gene of the MS part chromosome;

Step3: Repeat Step2 until all operations of all jobs are completed.

(2) Generation of OS part chromosomes.

The chromosomes of the OS part are generated using a random method.

3.4. Elite retention strategy

In the biological populations of nature, excellent individuals can often get more reproductive rights, so that the excellent genes in the population can be better inherited to the offspring. Inspired by the above-mentioned elite retention strategy (Chen, Ihlow, & Lehmann, 1999), a certain number of individuals with the best fitness are retained in each iteration.

3.5. Selection operator

Tournament selection method (Yang & Soh, 1997) is a common selection method. As early as 1991, Goldberg and Deb (1991) proved that the tournament selection method has better convergence and lower computational complexity than other selection operators. The tournament selection method randomly selects k individuals from the population each time ($k = 2$), and selects an individual with the best fitness value from these selected individuals, and repeats the above steps until the number of selected individuals meets the predetermined number scale. Due to the possibility that the tournament selection method fails to select the best contemporary individual, this paper designs a hybrid selection method that combines the tournament selection method and the elite retention to solve this problem.

The content of the hybrid selection method is as follows: When the number of parent individuals selected to participate in the crossover is N, (1- P_e)*N individuals are selected using the tournament selection method, and P_e*N individuals are selected using elite retention (P_e is the proportion of elite individuals). Therefore, there are a total of N individuals participating in the crossover. This hybrid selection method can not only improve the efficiency of algorithm selection, but also improve the convergence of the algorithm and reduce the complexity of the algorithm.

3.6. Combined crossover operator

The crossover operator determines the global search ability of the GA, and the crossover operator has a great influence on the performance of the genetic algorithm. Different crossover operators have different characteristics. If the combined crossover operators formed by multiple crossover operators are used in the crossover process, the characteristics of different crossover operators can be used to improve the global search ability of the algorithm. In addition, the combined crossover operator can ensure that the sequence of each gene remains unchanged, and that the processing sequence of the operations contained in each job is not destroyed. Segment coding includes two parts, namely MS part and OS part. In MS part, two different crossover operators are used according to the probability P_{cc}, which can enhance the global search capability of the algorithm.

(1) MS part.

The crossover operator must ensure that the sequence of genes on the chromosome remains unchanged, and the MS part can use 0–1 crossover operator and fixed position crossover operator.

1) 0–1 crossover operator.

The specific steps of the 0–1 crossover operator are as follows:

Step1: Randomly select one number in the set {0,1};

Step2: Perform the operations in Step1 T_0 times, and finally generate a 0–1 set key of length T_0;

Step3: Select the position corresponding to the number 1 in the key of the MS gene;

Step4: Exchange the gene at the selected position of the parent chromosome.

In order to facilitate the description of the 0–1 crossover operator, take the two parent chromosomes P_1 and P_2 participating in the crossover as an example. The two offspring chromosomes produced by the 0–1 crossover operator through crossover are C_1 and C_2. Fig. 5 shows the specific process of 0–1 crossover.

In Fig. 5, since key = [0 1 0 0 1 0 1], the values corresponding to the second, fifth, and seventh positions in P_1 and P_2 are copied to the corresponding positions in C_2 and C_1, respectively, and the values of the remaining positions remain unchanged.

2) Fixed position crossover (FPX) operator.

The specific steps of the fixed position crossover (FPX) operator are as follows:

Step1: Randomly generate a 0–1 array, requiring that 0 and 1 are not adjacent;

Step2: Exchange the value of the chromosome corresponding to the number 1.

The specific process diagram of 0–1 fixed position crossing is shown

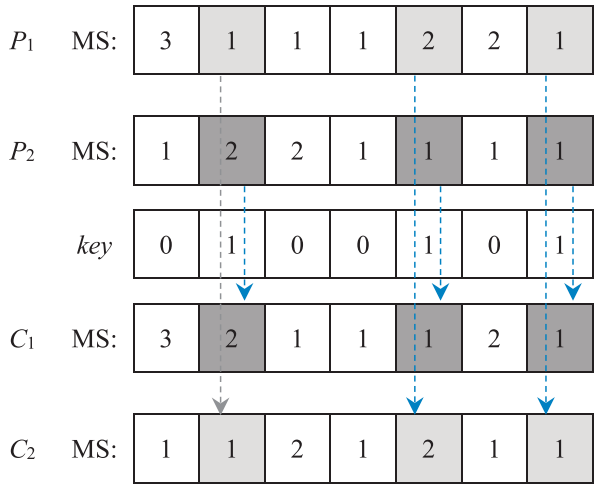


Fig. 5. 0–1 Crossover.

in Fig. 6.

(2) OS part.

The OS part uses the improved precedence operation crossover (IPOX) operator. Assuming the number of workpieces is n, the specific steps of the IPOX crossover operator are as follows:

Step1: Randomly generate an integer k in the interval [1, n-1], and generate k unequal integers in the interval [1, k];

Step2: k integers randomly divide the job set $\{J_1, J_2, \ldots, J_n\}$ into two job sets GJ_1 and GJ_2;

Step3: Copy the jobs contained in the parent chromosomes P_1 and P_2 in GJ_1 and GJ_2 to the offspring chromosomes C_1 and C_2 without changing their position and order;

Step4: Copy these jobs whose paternal chromosomes P_1 and P_2 are not included in the job sets GJ_1 and GJ_2 to C_2 and C_1, respectively, without changing their position and order.

In order to facilitate the description of the IPOX operator, we select the parent chromosomes P_1 and P_2 to crossover as an example. Randomly generate two unequal integers 2 and 4, and divide the jobs set into two parts, GJ_1={2, 4} and GJ_2={1, 3, 5}. Copy the jobs J_2, J_4 in P_1 to the corresponding positions in C_1, and fill the gaps from J_1, J_3, J_5 in P_2 to the remaining positions in C_1 according to their order in P_2; in the same way, copy the jobs J_2 and J_4 in P_2 to the corresponding positions in C_2, and the vacant positions are filled with J_1, J_3, and J_5 in P_1 to the remaining positions in C_1 according to their order in P_1. The schematic diagram of the two offspring produced by the crossover of the IPOX operator is shown in Fig. 7.

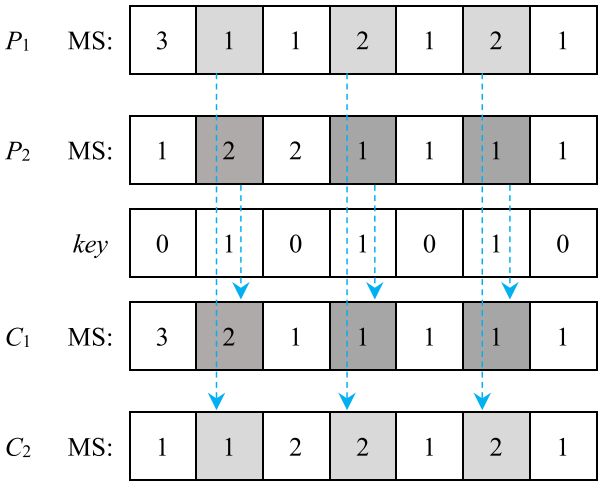


Fig. 6. Schematic diagram of FPX operator.

3.7. Combined mutation operator

The mutation operator can increase the diversity of the population, reduce the probability of premature convergence of the algorithm, and enhance the local search ability of the algorithm. Different mutation operators have different characteristics. If multiple mutation operators are used to construct a combined mutation operator in the mutation process, the characteristics of different mutation operators can be used to improve the local search ability of the algorithm. In order to improve the local search capability of the algorithm, we adopt a combined mutation operator. We use two mutation operators in the MS part according to the set probability P_{cm}, and only use a single operator in the OS part.

(1) MS part.

In the MS part, we use two mutation operators.

1) The first mutation operator.

The specific steps of the first mutation operator are as follows:

Step1: Select the operations with more than 1 optional processing machines (assuming there are t), and use their position numbers in the MS part as a set S={1, 2, …, t};

Step2: In the interval [1, t], randomly generate two unequal integers, R_1 and R_2;

Step3: Randomly replace the machine serial numbers at R_1 and R_2 with the serial numbers of other machines in the operations optional processing machine set at that position.

In order to facilitate the description of the first mutation operator, we take the FJSP in Table 1 as an example. Assume that two unequal integers generated randomly are $R_1 = 2$ and $R_2 = 6$. The operation corresponding to $R_1 = 2$ is O_{12}. In the optional machine set of the second operation of job 1, a processing machine is randomly selected, and its serial number is 2. The operation corresponding to $R_2 = 6$ is O_{31}. In the optional machine set of the first operation of job 3, a processing machine is randomly selected, and its serial number is 1. Use the obtained serial numbers 2 and 1 to replace the genes at the 2nd and 6th positions in the parent chromosome P_1, respectively, and the resulting offspring individual is C_1. The schematic diagram of the first mutation operator to generate offspring is shown in Fig. 8.

2) The second mutation operator.

Step1: In the interval [1, T_0], randomly generate r unequal integers, and treat them as r positions on the chromosome;

Step2: Select the positions from left to right, and replace the machines in each position with the machine with the shortest processing time of the currently available processing machines.

(2) OS part.

The OS part uses the exchange mutation operator, and the specific steps are as follows:

Step1: In the interval [1, T_0], randomly generate two unequal ordered integers R_1 and R_2;

Step2: Swap the numbers at the R_1 and R_2 positions on the chromosome.

In order to facilitate the description of the exchange mutation operator, the following examples are given. Assuming that the generated 2 unequal integers are 2 and 5, swapping the genes at positions 2 and 5, the resulting offspring chromosome is C_1. The schematic diagram of the exchange mutation operator is shown in Fig. 9.

3.8. Variable neighborhood search

In the FJSP, the longest path from the start point to the end point is

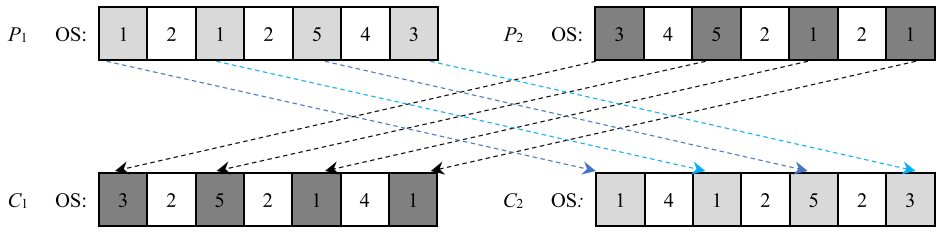


Fig. 7. Schematic diagram of IPOX operator.

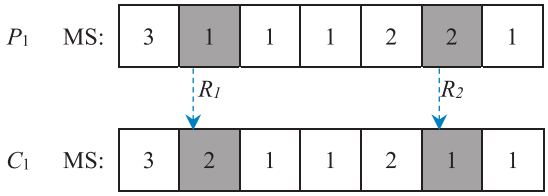


Fig. 8. Schematic diagram of the first mutation operator.

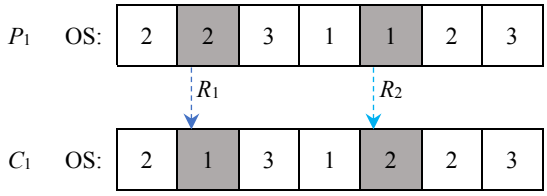


Fig. 9. Schematic diagram of the exchange mutation operator.

called the critical path, and its length is the makespan of the scheduling scheme. The length of the critical path determines the pros and cons of the scheduling scheme. In previous studies, some algorithms did not operate on the critical process on the critical path and performed a large number of invalid transformations, resulting in slow convergence and poor solution accuracy. Therefore, this paper adopts the neighborhood search method to transform the key process on the critical path, and conducts a detailed search for the optional machines of the key process to find the optimal machine combination, which improves the probability of the optimal solution.

(1) Find key operation.

The specific steps to find key operation are as follows:

Step1: Find the latest completion operation O_h, which is a key operation;

Step2: Find the completion time $C(PJ(h))$ of the previous operation $PJ(h)$ of the same job in the operation O_h and the completion time $C(PM(h))$ of the previous operation $PM(h)$ of the same machine;

Step3: If $C(PJ(h)) \geq C(PM(h))$, take $PJ(h)$ as the key operation, otherwise, take $PM(h)$ as the key operation;

Step4: Return to **Step1**, until an operation with a start time of 0 appears, and record all key operation.

The pseudo code for finding key operation is shown in **Algorithm 1**.

Algorithm 1: Find key operation

1: Begin
2: Find the latest completion operation O_h;
3: Find the completion time $C(PJ(h))$ of the previous operation $PJ(h)$ of the same job in the operation O_h and the completion time $C(PM(h))$ of the previous operation $PM(h)$ of the same machine;
4: If $C(PJ(h)) \geq C(PM(h))$
5: Take $PJ(h)$ as the key operation;
6: else
7: take $PM(h)$ as the key operation;

(*continued on next column*)

(*continued*)

Algorithm 1: Find key operation

8: End if
9: Repeat the above steps until an operation with a start time of 0 appears, record all the key operations, and the critical path.

(2) Variable neighborhood search.

In general, the variable neighborhood search process keeps the OS part of the key operation chromosome unchanged, and constantly changes the MS part according to the law, and then compares the newly combined chromosome decoding with the original chromosome. If the new chromosome is better than the original chromosome, then replace the original chromosome. The pseudo code of the variable neighborhood search is shown in **Algorithm 2**. The transformation process of the variable neighborhood search can be seen in Fig. 10.

Algorithm 2:Variable neighborhood search (VNS)

1: Begin
2: The MS part and OS part of chromosome k are $Xmachine_k$ and $Xprocess_k$, and the number of key operations is L;
3: For $i = 1$: L
4: Establish a set of the h-th operation on the critical path;
5: Record the processing machine of the current operation best_machine=$Xmachine_k$;
6: For $j = 1$: L
7: $Xmachine_{k,h,i}$=$best\_machine_{h,j}$;
8: Calculate the fitness value of the new chromosome combined with $Xmachine_{k,h,j}$ and $Xprocess_k$;
9: If fit($Xmachine_{k,h,j}$) < fit(best_machine$_{h,j}$)
10: $best\_machine$=$Xmachine_{k,h,j}$;
11: End if
12: End for
13: End for

3.9. The framework of the proposed algorithm

The specific steps of the hybrid genetic algorithm with variable neighborhood search (HGA-VNS) are as follows:

Step1: The parameter settings of the algorithm include the population size N, the k value in the tournament selection section, the proportion of elite individuals P_e and other parameters;

Step2: Encoding and population initialization;

Step3: Decode and evaluate the fitness values of the individuals in the population, and sort them. The retained elite individuals and the individuals selected by the tournament selection method together form a new population;

Step4: Perform combined crossover operation with P_c, and the probability of choosing 0–1 crossover is P_{cc};

Step5: Perform combined mutation operation with P_m, and the probability of selecting the first mutation operator is P_{mm};

Step6: Perform variable neighborhood search;

Step7: Return to **step 3** until the maximum number of iterations is reached.

The pseudo code of the HGA-VNS algorithm is shown in **Algorithm**

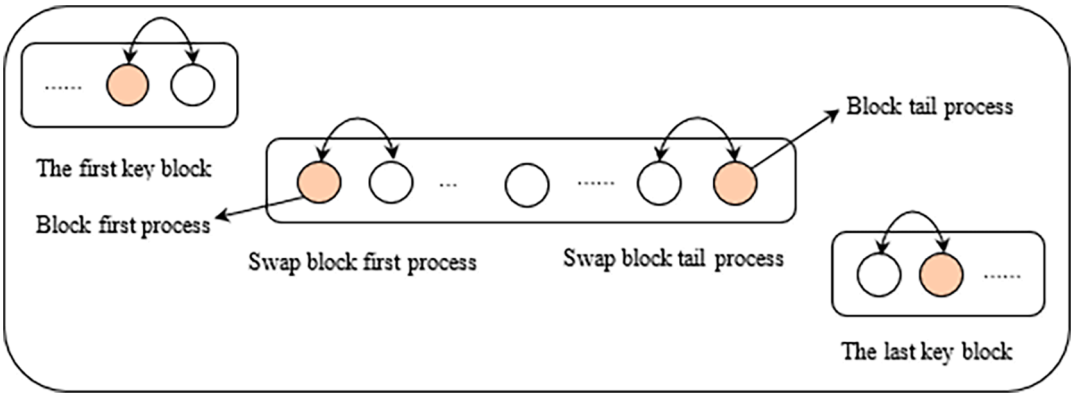


Fig. 10. The transformation process of the neighborhood structure.

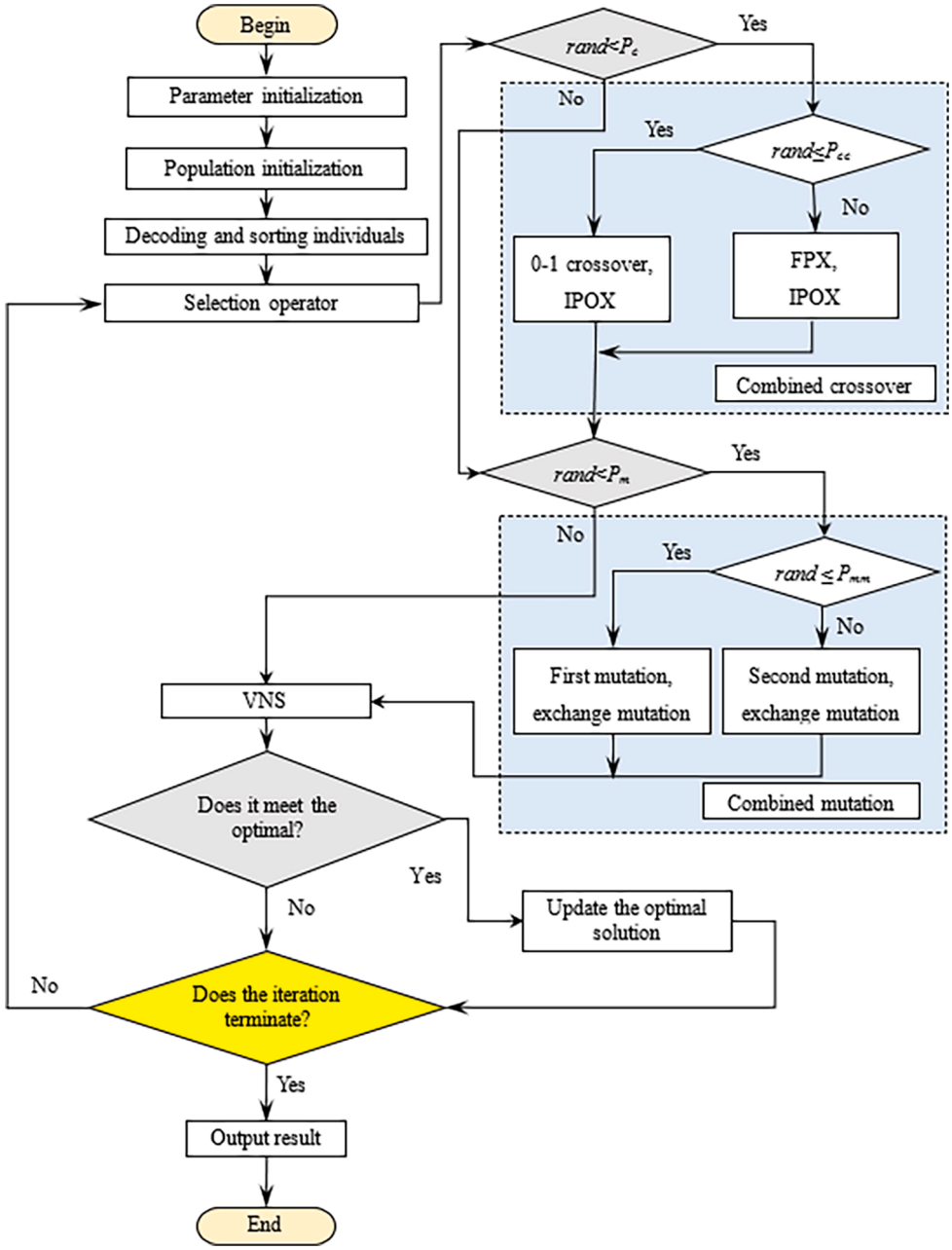


Fig. 11. Flow chart of the HGA-VNS.

3, and the framework of the HGA-VNS algorithm is shown in Fig. 11.

Algorithm 3: HGA-VNS

1: Begin
2: Parameter initialization setting;
3: Encoding and population initialization;
4: The elite retention strategy and the tournament selection method are used to select the parent individuals who participate in the crossover;
5: Combined crossover operation;
6: Combined mutation operation;
7: Find the critical path and perform variable neighborhood search;
8: Judge whether it reaches the maximum number of iterations, if it is satisfied, stop and output the result, otherwise return to Selection operator;
9: End

3.10. The analysis of the computational complexity

In our proposed HGA-VNS, it contains encoding and decoding scheme, population initialization, selection operator, combined crossover operator, combined mutation operator, and variable neighborhood search. Let N be the population size, T_0 be the total number of operations of all jobs, m be the number of machines, and L be the number of key operations. In the HGA-VNS, the computational complexity of population initialization is $O(N*T_0)$, the computational complexity of selection operator is $O(N)$, the computational complexity of combined crossover operator is $O(N*T_0)$, the computational complexity of combined mutation operator is $O(N*T_0)$, and the computational complexity of VNS is $O(N*L^2)$. Therefore, the total computational complexity of the HGA-VNS is $O(N*T_0)$.

By analyzing some other literature (Serna, Seck-Tuoh-Mora, Medina-Marin, Hernandez-Romero, & Armenta, 2021; Chen, Yang, Li, & Wang, 2020; Zhang, Zhang, Song, Wang, & Zhou, 2019; Zarrouk, Bennour, & Jemai, 2019; Buddala & Mahapatra, 2019), we can know the corresponding computational complexity of these algorithms. The computational complexity of HGA-VNS and some other algorithms are summarized in Table 3. It can be seen from Table 3 that the total computational complexity of the HGA-VNS is not worse than other algorithms.

4. Numerical experiment

4.1. Experimental evaluation method

Friedman test (Derrac et al., 2011) is a multiple comparison test that can be used to observe whether there are significant differences between two or more algorithms. This method first converts the original data into a ranking, and then obtains the final ranking of the algorithm. Its specific implementation process is as follows:

Step1: Calculate the solution results of various algorithms for each calculation example;

Step2: According to the optimal value of each algorithm, the algorithms are sorted from small to large, and the ranking of the j-th objective function by the i-th algorithm is recorded as $r_{ij}(1 \le i \le m, 1 \le j \le n)$;

Step3: Calculate the average R_i of the rank ranking of each algorithm corresponding to each test function. The calculation formula of R_i is as follows:

$$R_i = \frac{1}{n}\sum_{j=1}^{n} r_{ij} (i = 1, 2, \cdots, m) \tag{5}$$

Step4: According to the average ranking of each algorithm in all test functions, the final ranking of various algorithms is obtained.

Analysis of the obtained results shows that the smaller the rank of the algorithm, the better the performance of the algorithm. In other words, the ranking of the best performing algorithm should be 1, followed by 2, and so on.

Although Friedman's rank ranking gives the ranking of the results obtained by different algorithms, it does not indicate that the performance differences between the algorithms are significant. Therefore, it is also necessary to perform Friedman's test on the rank ranking results obtained. The specific implementation process of Friedman's test is as follows:

(1) At the significance level α, a one-sided test is performed. Assume as follows:

H_0: There is no obvious difference in the average rank ranking obtained by the algorithms participating in the comparison;
H_1: There are obvious differences in the average rank ranking obtained by the algorithms participating in the comparison;

(2) Calculate the rank corresponding to the average objective function value of each algorithm in each function, assign the optimal value of the average objective function value to rank 1, and assign the worst value of the average objective function value to k.
(3) The statistic of Friedman's test is calculated as follows:

$$\chi^2 = \frac{12n}{k(k+1)}\left[\sum_{j=1}^{k}(R_j)^2 - \frac{k(k+1)^2}{4}\right] \tag{6}$$

R_j is the average rank of the j-th algorithm in the overall algorithm, k is the number of algorithms, and n is the number of calculation examples.

(4) According to the pre-set significance level α and the degree of freedom k-1, the critical value $\chi^2_{\alpha(k-1)}$ of the chi-square distribution can be obtained by querying the chi-square distribution table. If $\chi^2 > \chi^2_{\alpha(k-1)}$, then reject the null hypothesis and prove that the average ranks obtained by the algorithms participating in the comparison are obviously different; otherwise, accept the null hypothesis and prove the average rank obtained by the algorithms participating in the comparison is not significantly different.

Iman, Davenport, and i. S.-T., & Methods. (1980) gave a better statistic F_{ID} based on Friedman's test. In order to more fully prove that there are significant differences in the performance of the algorithm, the two statistics of χ^2 and F_{ID} are used at the same time when the non-parametric test is performed. The calculation formula of F_{ID} statistics

Table 3
The computational complexity of HGA-VNS and some other algorithms.

Literature	Initialization	Operator 1	Operator 2	Operator 3	Local search	Total complexity
Buddala and Mahapatra (2019)	$O(N*T_0)$	$O(2*N)$	$O(2*N)$	/	/	$O(N*T_0)$
Zarrouk et al. (2019)	$O(N*T_0)$	$O(4*N)$	/	/	$O(N*m*T_n)$	$O(N*m*T_n)$
Zahng, et al. (2019)	$O(N*T_0)$	$O(3*N)$	/	/	$O(N*m*T_n)$	$O(N*m*T_n)$
Chen et al. (2020)	$O(N*T_0)$	$O(5*N)$	/	/	/	$O(N*T_0)$
Serna et al. (2021)	$O(N*T_0)$	$O(N_s)$	$O(N)$	/	$O(N_s*m*T_n)$	$O(N_s*m*T_n)$
This paper	$O(N*T_0)$	$O(N)$	$O(N*T_0)$	$O(N*T_0)$	$O(N*L^2)$	$O(N*T_0)$

Note: N_s is the smart-cells number and T_n is tabu iterations number in literature (Serna et al., 2021).

is as follows:

$$F_{ID} = \frac{(n-1)\chi^2}{n(k-1)-\chi^2} \tag{7}$$

Similarly, you can query the F distribution table to obtain the value of $F_{[(k\text{-}1),\,(k\text{-}1)(n\text{-}1)]}$ according to the preset α, and then compare it with F_{ID}. According to the result of the comparison, it is judged whether there is a significant difference in the performance of the algorithm.

4.2. Investigation of the optimal parameter combination

In the intelligent optimization method, different parameter values have a great influence on the performance of the algorithm. In the proposed HGA-VNS algorithm, the important parameters that have a greater impact on the performance of the algorithm are the crossover probability P_c, the mutation probability P_m, the proportion of elite individuals P_e, the selection probability of the combined crossover operator P_{cc}, and the selection probability of the combined mutation operator P_{mm}. Crossover probability P_c greater than 0.6 can better explore the solution space (Meziane & Taghezout, 2018), and the value of P_c in the current literature (Driss et al., 2015; Ishikawa, Kubota, & Horio, 2015; Wang, Yin, & Qin, 2013) is usually 0.8. For other parameters of the proposed HGA-VNS algorithm, the orthogonal experiment in Table 4 is designed to find the best combination of parameters through experiments. Table 4 summarizes the specific values of the orthogonal design scheme.

In the orthogonal experiment, the maximum number of iterations of the algorithm is set to 500, and the population size is set to 100. At the same time, we choose some test cases proposed by Brandimarte (1993) as test functions. Due to the large number of experimental programs in the orthogonal experiments designed in this paper, and each group of experiments contains 10 test cases, it is difficult to directly judge which group of experimental programs has the best effect. In order to better compare the pros and cons of the parameter combinations in various experimental schemes, the Friedman rank ranking is performed on the results obtained by the algorithm, and the significance test of the rank ranking results is performed. Table 5 summarizes the average calculation results of the orthogonal experiments. Table 6 shows the ranking of each group of experiments to solve different examples. Based on this, the average ranking of each group of experiments is calculated, and then the final ranking is obtained.

The results of mean rank and final ranking for various experimental schemes are shown in Fig. 12. As shown in Fig. 12, the average rank and final ranking of Experiment 4 are significantly better than that of other experimental schemes. In other words, when $P_m = 0.3$, $P_{cc} = 0.3$, $P_{mm} = 0.5$, $P_e = 0.1$, the performance of the HGA-VNS algorithm is optimal compared to other experimental schemes.

The results of Friedman statistic χ^2 on orthogonal experiment scheme are shown in Table 7. According to Table 7, we can know that the degree of freedom df is 8, and the calculated result of Friedman statistic χ^2 is 22.387. When the significance level α is 0.05, by querying the chi-square distribution table, it can be seen that the critical value $\chi^2_{\alpha(k-1)}$ of the chi-square distribution is 15.507. Because $\chi^2 = 22.387 > \chi^2_{\alpha(k-1)} = 15.507$, reject the null hypothesis and accept the opposite hypothesis. When the significance level α is 0.01, by querying the chi-square distribution table, it can be seen that the critical value $\chi^2_{\alpha(k-1)}$ of the chi-square distribution is 20.090. Because $\chi^2 = 22.387 > \chi^2_{\alpha(k-1)} = 20.090$, reject the null hypothesis and accept the opposite hypothesis. The results of Friedman's test in $\alpha = 0.05$ and 0.01 show that there are obvious differences in the average rank obtained by orthogonal experiment scheme.

Table 4
Four-factor three-level orthogonal experiment.

Experiment number	P_m	P_{cc}	P_{mm}	P_e
1	0.1	0.3	0.3	0.06
2	0.1	0.5	0.5	0.08
3	0.1	0.7	0.7	0.1
4	0.3	0.3	0.5	0.1
5	0.3	0.5	0.7	0.06
6	0.3	0.7	0.3	0.08
7	0.5	0.3	0.7	0.08
8	0.5	0.5	0.3	0.1
9	0.5	0.7	0.5	0.06

The results of Friedman statistic F_{ID} on orthogonal experiment scheme are shown in Table 8. The calculated result of Friedman statistic F_{ID} is 3.497. Because k-1 = 8, $(k\text{-}1)(n\text{-}1) = 72$ and $\alpha = 0.05$, $F_{(8,72)}$ is distributed according to F distribution with degrees of freedom 7 and 72, the critical value of $F_{(8,72)}$ for $\alpha = 0.05$ is 2.070. Because $F_{ID} = 3.497 > 2.070$, the null hypothesis is rejected in $\alpha = 0.05$. Similarly, when $\alpha = 0.01$ and $F_{(8,72)}$ is distributed according to F distribution with degrees of freedom 7 and 72, the critical value of $F_{(8,72)}$ for $\alpha = 0.01$ is 2.769. Because $F_{ID} = 3.497 > 2.769$, the null hypothesis is rejected in $\alpha = 0.01$. The results of Friedman test in $\alpha = 0.05$ and 0.01 show that there are obvious differences in the average ranking obtained by orthogonal experiment scheme.

From the analysis in Fig. 12, Tables 7 and 8, there are significant differences in the performance of orthogonal experiment scheme, and the parameter combination of Experiment 4 is the best among all the parameter combinations in the orthogonal experiment. Therefore, the value of the parameter combination in Experiment 4 is used as the parameter value of the proposed HGA-VNS algorithm.

4.3. Comparison of HGA-VNS with other GA variants

In the HGA-VNS, the parameters of the experiment are set as follows: the crossover probability P_c is 0.8, and the remaining parameters ($P_m = 0.3$, $P_{cc} = 0.3$, $P_{mm} = 0.5$, $P_e = 0.1$) are obtained according to the orthogonal experiment to obtain the best parameter values. The maximum number of iterations is adjusted according to the complexity of the test problem to ensure that a relatively high-quality solution can be obtained. Table 9 shows the population size and the maximum number of iterations of each algorithm. Each algorithm uses the maximum number of iterations as the termination condition of the program.

In order to verify the effectiveness of the algorithm improvement, the hybrid genetic algorithm with variable neighborhood search (HGA-VNS) is compared with the genetic algorithm (GA) and the ordinary genetic algorithm with variable neighborhood search (GA-VNS). HGA-VNS, GA-VNS and GA, except for some operation operators are different, other operation operators are the same, and the parameters remain the same. The difference between the operation operators of HGA-VNS, GA-VNS and GA is shown in Table 10. For different algorithms, the same population size and the same number of iterations are set, and the test cases selects the FJSP designed by Brandimarte. The algorithms involved in the comparison run 10 times independently during the comparison. The solution results of the algorithms participating in the comparison are recorded.

The average value of the maximum completion time obtained by the algorithms participating in the comparison under each test case is shown in Table 11. As seen from Table 11, the average optimal values obtained by the HGA-VNS with five cases are better than the other algorithms involved in the comparison, and the average optimal values obtained by the HGA-VNS with three cases are equal to the average optimal values obtained by the GA-VNS algorithm.

Fig. 13 shows that some convergence curves of three algorithms (HGA-VNS, GA-VNS and GA) on test cases, where the x-axis represents the number of iterations and the y-axis represents the makespan in each subgraph. It can be seen from Fig. 13 that among the three algorithms, HGA-VNS has the best convergence, GA-VNS has the second-best convergence, and ordinary GA has the worst convergence. The convergence results in Fig. 13 show that the HGA-VNS algorithm can get a

Table 5
The average calculation results of orthogonal experiments scheme.

Test cases	1	2	3	4	5	6	7	8	9
mk01	**40.1**	40.5	40.5	40.3	40.4	40.3	40.7	40.5	40.4
mk02	27.8	27.4	27.4	27.1	26.9	27	26.9	26.9	**26.7**
mk03	204	204	204	204	204	204	204	204	204
mk04	65.1	65.6	63.9	63	63.1	63	63.7	62.4	**62.1**
mk05	174.3	174.5	174.3	173.5	**172.9**	173.3	173.2	173.1	173.1
mk06	67.7	67.1	66.5	64.8	65.4	**63.7**	65.6	65.2	65.2
mk07	142.8	143	143.1	140.9	140.9	141.4	**140.3**	140.5	141
mk08	523	523	523	523	523	523	523	523	523
mk09	318.5	320.4	320.4	313.1	314	**312.5**	317.6	318.5	317.6
mk10	225.3	228.9	229.8	**219**	221.9	219.6	235.1	233.4	234.7

Table 6
The Friedman rank of orthogonal experiments scheme.

Test cases	1	2	3	4	5	6	7	8	9
mk01	1	7	7	2.5	4.5	2.5	9	7	4.5
mk02	9	7.5	7.5	6	3	5	3	3	1
mk03	5	5	5	5	5	5	5	5	5
mk04	8	9	7	3.5	5	3.5	6	2	1
mk05	7.5	9	7.5	6	1	5	4	2.5	2.5
mk06	9	8	7	2	5	1	6	3.5	3.5
mk07	7	8	9	3.5	3.5	6	1	2	5
mk08	5	5	5	5	5	5	5	5	5
mk09	6.5	8.5	8.5	2	3	1	4.5	6.5	4.5
mk10	4	5	6	1	3	2	9	7	8

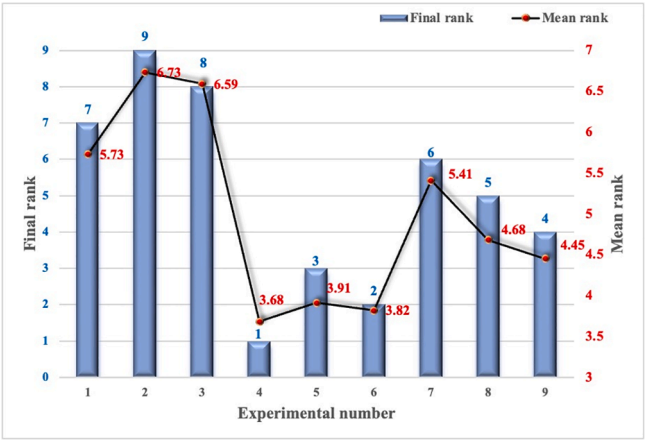


Fig. 12. Mean rank and final rank of each experiment scheme.

Table 7
The results of Friedman statistic χ^2 in different significance level.

k	df	α	χ^2	$\chi^2_{\alpha(k-1)}$	H_0	H_1
9	8	0.05	22.387	15.507	×	√
9	8	0.01	22.387	20.090	×	√

Table 8
The results of Friedman statistic F_{ID} in different significance level.

k	df	α	F_{ID}	$F_{[(k-1),\ (k-1)(n-1)]}$	H_0	H_1
9	8	0.05	3.497	2.070	×	√
9	8	0.01	3.497	2.769	×	√

better solution in the least number of iterations. Figs. 14 and 15 are the Gantt charts of the optimal scheduling schemes obtained by solving the mk02 and mk04 examples using the HGA-VNS algorithm.

According to the solution results in Table 12, it cannot be explained that the performance of the HGA-VNS is excellent. In order to compare the pros and cons of the three algorithms, the solution results of the algorithms are ranked by Friedman rank, and the performance differences of all algorithms are tested whether the difference is significant. The results in Table 12 are the results obtained by using the Friedman rank method to perform the Friedman ranking on the results of the three algorithms in Table 11.

In Table 12, the final rank of HGA-VNS is the first, but this does not indicate that there are significant differences in the performance of the algorithms participating in the comparison. In order to show that there are significant differences in the performance, we performed Friedman significance test.

The results of Friedman statistic χ^2 on 10 test cases for three algorithms (GA, GA-VNS and HGA-VNS) are shown in Table 13. According to Table 13, we can know that the degree of freedom df is $= 3$, and the calculated result of Friedman statistic χ^2 is 10.050. When the significance level α is 0.05, by querying the chi-square distribution table, it can be seen that the critical value $\chi^2_{\alpha(k-1)}$ of the chi-square distribution is 5.991. Because $\chi^2 = 10.050 > \chi^2_{\alpha(k-1)} = 5.991$, reject the null hypothesis and accept the opposite hypothesis. When the significance level α is 0.01, by querying the chi-square distribution table, it can be seen that the critical value $\chi^2_{\alpha(k-1)}$ of the chi-square distribution is 10.050. Because $\chi^2 = 10.050 > \chi^2_{\alpha(k-1)} = 9.210$, reject the null hypothesis and accept the opposite hypothesis. The results of Friedman test in $\alpha = 0.05$ and 0.01 show that there are obvious differences in the average rank obtained by GA, GA-VNS and HGA-VNS.

The results of Friedman statistic F_{ID} on 10 test cases for three algorithms (GA, GA-VNS and HGA-VNS) are shown in Table 14. The calculated result of Friedman statistic F_{ID} is 9.091. Because k-1 = 2, $(k\text{-}1)(n\text{-}1) = 18$ and $\alpha = 0.05$, $F_{(2,18)}$ is distributed according to F distribution with degrees of freedom 2 and 18, the critical value of $F_{(2,18)}$ for $\alpha = 0.05$ is 3.555. Because $F_{ID} = 9.091 > 3.555$, the null hypothesis is rejected in $\alpha = 0.05$. Similarly, when $\alpha = 0.01$ and $F_{(2,18)}$ is distributed according to F distribution with degrees of freedom 2 and 18, the critical value of $F_{(2,18)}$ for $\alpha = 0.01$ is 2.769. Because $F_{ID} = 9.091 > 6.013$, the null hypothesis is rejected in $\alpha = 0.01$. The results of Friedman test in $\alpha = 0.05$ and 0.01 show that there are obvious differences in the average ranking obtained by GA, GA-VNS and HGA-VNS.

From the analysis in Fig. 13 and Tables 11–14, the solution result of HGA-VNS is obviously better than that of other algorithms participating in the comparison, and there are significant differences in the performance of the three algorithms participating in the comparison. The performance of GA-VNS is better than that of GA, which shows that variable neighborhood search can improve the algorithm's solving ability; the performance of HGA-VNS is better than that of GA-VNS and GA, which shows that combined crossover operator and combined mutation operator can improve the algorithm's performance.

4.4. Comparison of HGA-VNS with other improved algorithms

In order to further verify the effectiveness of the HGA-VNS for

Table 9
The population size and the maximum number of iterations of each algorithm.

Test cases	mk01	mk02	mk03	mk04	mk05	mk06	mk07	mk08	mk09	mk10
Population	300	200	200	300	300	300	300	200	500	500
Iteration	500	500	500	500	500	500	500	500	700	700

Table 10
The difference between the operation operators of HGA-VNS, GA-VNS and GA.

Operation operators		GA	GA-VNS	HGA-VNS
Crossover	MS	0–1 crossover	0–1 crossover	0–1 crossover and FPX
	OS	IPOX	IPOX	IPOX
Mutation	MS	First mutation	First mutation	First and second mutation
	OS	exchange mutation	exchange mutation	exchange mutation
LS		–	VNS	VNS

Note: MS stands for machine selection, OS stands for process sorting, and LS stands for local search.

Table 11
The average optimal values obtained by three algorithms.

Test cases	GA	GA-VNS	HGA-VNS
mk01	41.7	40.2	**40**
mk02	28	26.9	**26.6**
mk03	204	204	204
mk04	66	**61.2**	61.4
mk05	175.9	**173**	**173**
mk06	68	**63.1**	63.5
mk07	144.2	140.6	**140.3**
mk08	523	523	523
mk09	318.1	309.3	**309.1**
mk10	232.4	217.2	**216.9**

solving FJSPs, the HGA-VNS is compared with the algorithms in other references. The algorithm involved in the comparison is run 10 times, and the best solution is selected as the optimal solution. We compare the optimal value obtained by the HGA-VNS with a variety of other improved algorithms, such as the hybrid gray wolf optimization algorithm (HGWO) proposed by Jiang (2018), the new immune multi-agent system algorithm (NIMASS) by Xiong and Fu (2018), the adaptive discrete cat swarm optimization algorithm (ADCSO) proposed by Jiang and Zhang (2019), the improved particle swarm optimization algorithm (PSO) (Ding & Gu, 2020b) and HLO-PSO algorithm (Ding & Gu, 2020a), and a self-learning genetic algorithm (SLGA) based on reinforcement learning proposed by Chen et al. (2020). The solution results of the algorithms participating in the comparison are shown in Table 15.

Note: n represents the number of jobs, m represents the number of machines, and the scale refers to the total number of operations.

In order to compare the performance of each algorithm, the relative percentage deviation (RPD) is defined. The calculation method of RPD is as follows:

$$RPD = \frac{BOV - BKV}{BOV} \times 100\% \tag{8}$$

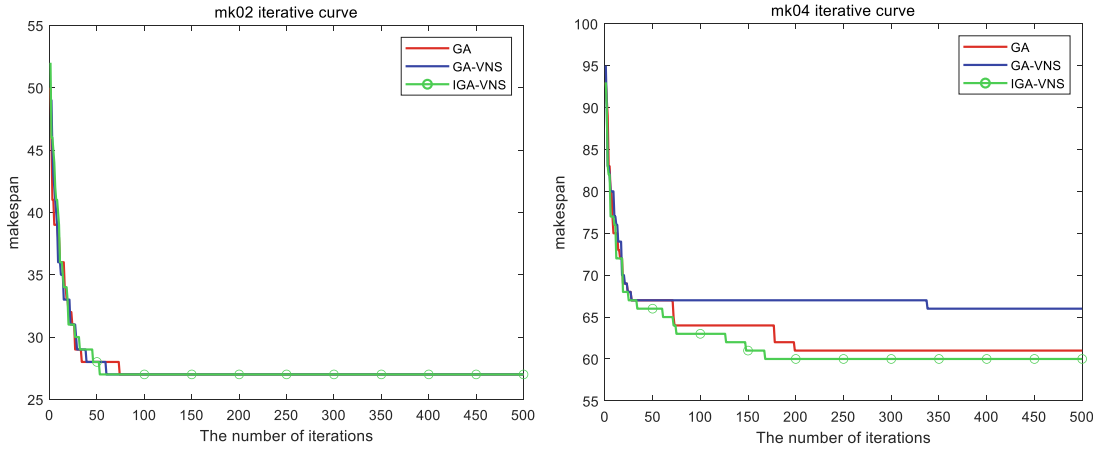
Where BOV represents the optimal value obtained by the algorithm, and BKV represents the known optimal value of each test case in the current research.

By analyzing Eq. (8), it can be seen that the larger the RPD value equation is, the larger the gap between the algorithm solution result and the known optimal value; on the contrary, the smaller the gap. The calculation results of RPD participating in the comparison algorithm are shown in Table 16. As can been seen from Table 16 that the average RPD value obtained by HGA-VNS is the smallest, which means that the optimal value obtained by HGA-VNS is closer to the known optimal value than the optimal value obtained by all other algorithms involved in the comparison.

Table 17 shows the Friedman rank of all the algorithms participating in the comparison. As can been seen in Table 17, when HGA-VNS solves mk02, mk06, mk07, mk09 and mk10, the results obtained are better than other algorithms involved in the comparison. When solving the remaining test cases, the worst ranking obtained by HGA-VNS is No. 4, which shows that the HGA-VNS algorithm is more excellent in solving accuracy.

The mean rank and final rank of the RPD obtained by the various algorithms involved in the comparison are shown in Fig. 16. As seen in Fig. 16, the final Friedman ranking of the RPD value sought by HGA-VNS is better than other algorithms participating in the comparison.

In order to better compare the performance differences of the participating algorithms, we conducted a Friedman test. The results of Friedman statistic χ^2 on 10 test cases for various improved algorithms



(a) Iterative convergence curve of mk02 (b) Iterative convergence curve of mk04

Fig. 13. The convergence curves of three algorithms on test cases.

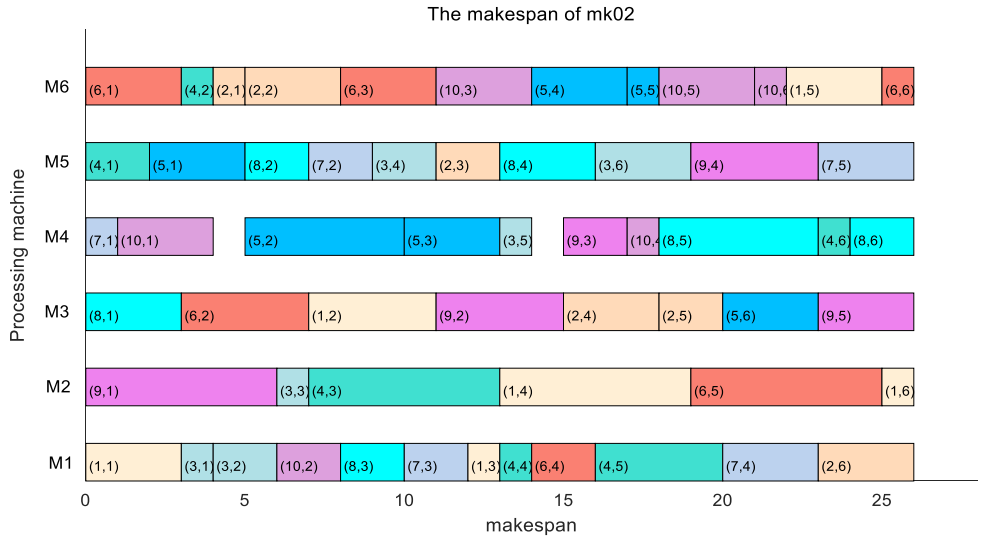


Fig. 14. The Gantt charts of the optimal scheduling schemes for mk02 by HGA-VNS.

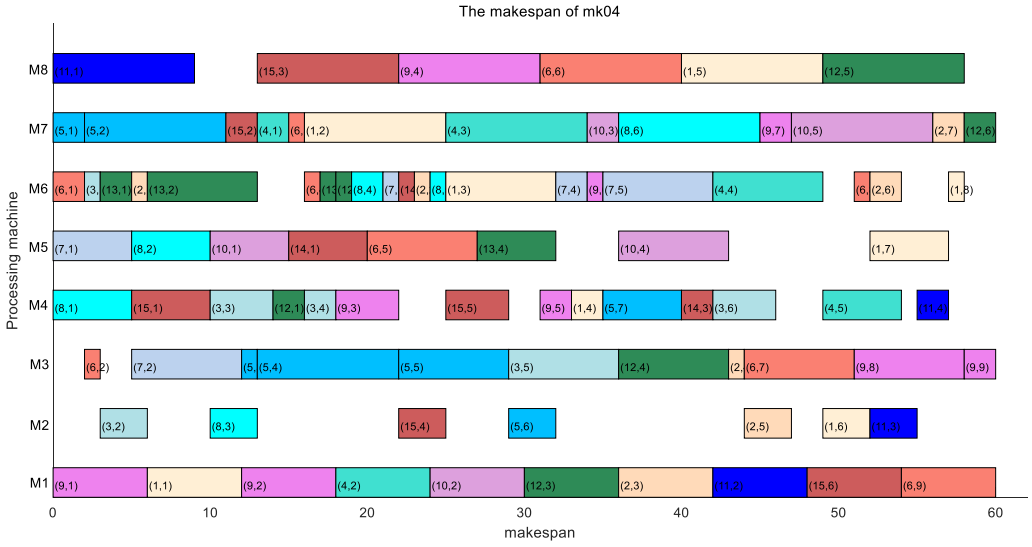


Fig. 15. The Gantt charts of the optimal scheduling schemes for mk04 by HGA-VNS.

Table 12
The mean rank and final rank of the three algorithms.

Test cases	GA	GA-VNS	HGA-VNS
mk01	3	2	1
mk02	3	2	1
mk03	2	2	2
mk04	3	1	2
mk05	3	1.5	1.5
mk06	3	1	2
mk07	3	2	1
mk08	2	2	2
mk09	3	2	1
mk10	3	2	1
Mean rank	2.8	1.75	1.45
Final rank	3	2	1

Table 13
The results of Friedman statistic χ^2 in different significance level.

k	df	α	χ^2	$\chi^2_{\alpha(k-1)}$	H_0	H_1
3	2	0.05	10.050	5.991	×	√
3	2	0.01	10.050	9.210	×	√

Table 14
The results of Friedman statistic F_{ID} in different significance level.

k	df	α	F_{ID}	$F_{[(k\text{-}1),\ (k\text{-}1)(n\text{-}1)]}$	H_0	H_1
3	2	0.05	9.091	3.555	×	√
3	2	0.01	9.091	6.013	×	√

(HGWO, NIMASS, ADCSO, PSO, HLO-PSO, SLGA and HGA-VNS) are shown in Table 18. According to Table 18, we can know that the degree of freedom df is 6, and the calculated result of Friedman statistic χ^2 is 18.311. When the significance level α is 0.05, by querying the chi-square distribution table, it can be seen that the critical value $\chi^2_{\alpha(k-1)}$ of the chi-square distribution is 12.592. Because $\chi^2 = 18.311 > \chi^2_{\alpha(k-1)} = 12.592$, reject the null hypothesis and accept the opposite hypothesis. When the significance level α is 0.01, by querying the chi-square distribution table, it can be seen that the critical value $\chi^2_{\alpha(k-1)}$ of the chi-square distribution is 16.812. Because $\chi^2 = 18.311 > \chi^2_{\alpha(k-1)} = 9.210$, reject the null

Table 15
The optimal solution obtained by the algorithms participating in the comparison.

Test cases	$n \times m$	Scale	HGWO	NIMASS	ADCSO	PSO	HLO-PSO	SLGA	HGA-VNS
mk01	10 × 6	55	40	40	40	40	40	40	40
mk02	10 × 6	58	29	28	27	29	28	27	**26**
mk03	15 × 8	150	204	204	204	204	204	204	204
mk04	15 × 8	90	65	65	64	66	63	**60**	**60**
mk05	15 × 4	106	175	177	173	175	175	**172**	173
mk06	10 × 15	150	79	67	66	77	71	69	**61**
mk07	20 × 5	100	149	144	144	145	144	144	**140**
mk08	20 × 10	225	523	523	523	523	523	523	523
mk09	20 × 10	240	325	312	311	320	326	320	**307**
mk10	20 × 15	240	253	229	226	239	238	254	**214**

Table 16
The results of RPD obtained by various algorithms.

Test cases	BKV	HGWO	NIMASS	ADCSO	PSO	HLO-PSO	SLGA	HGA-VNS
mk01	36	10.0	10.0	10.0	10.0	10.0	10.0	10.0
mk02	24	17.2	14.3	11.1	17.2	14.3	11.1	**7.7**
mk03	204	0.0	0.0	0.0	0.0	0.0	0.0	0.0
mk04	48	26.2	26.2	25.0	27.3	23.8	**20.0**	**20.0**
mk05	168	4.0	5.1	2.9	4.0	4.0	**2.3**	2.9
mk06	33	58.2	50.7	50.0	57.1	53.5	**52.2**	45.9
mk07	133	10.7	7.6	7.6	8.3	7.6	7.6	**5.0**
mk08	523	0.0	0.0	0.0	0.0	0.0	0.0	0.0
mk09	299	8.0	4.2	3.9	6.6	8.3	6.6	**2.6**
mk10	165	34.8	27.9	27.0	31.0	30.7	35.0	**22.9**
Mean		16.9	14.6	13.7	16.1	15.2	14.5	11.7

Table 17
The Friedman rank of all the algorithms participating in the comparison.

Test cases	HGWO	NIMASS	ADCSO	PSO	HLO-PSO	SLGA	HGA-VNS
mk01	4	4	4	4	4	4	4
mk02	6.5	4.5	2.5	6.5	4.5	2.5	**1**
mk03	4	4	4	4	4	4	4
mk04	5.5	5.5	4	7	3	**1.5**	**1.5**
mk05	5	7	2.5	5	5	1	2.5
mk06	7	3	2	6	5	4	**1**
mk07	7	3.5	3.5	6	3.5	3.5	**1**
mk08	4	4	4	4	4	4	4
mk09	6	3	2	4.5	7	4.5	**1**
mk10	6	3	2	5	4	7	**1**

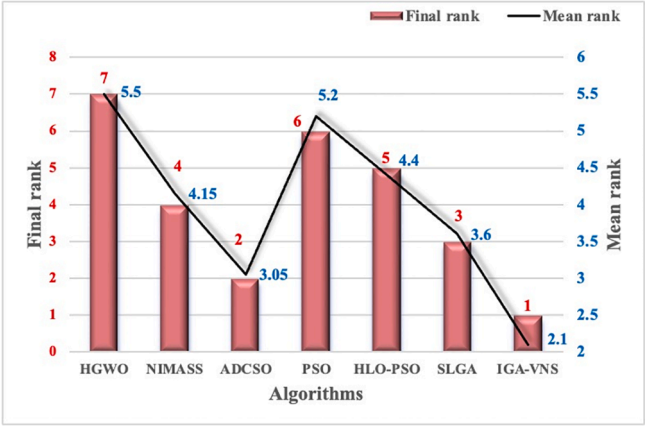


Fig. 16. Mean rank and final rank of RPD obtained by each algorithm.

hypothesis and accept the opposite hypothesis. The results of Friedman test in $\alpha = 0.05$ and 0.01 show that there are obvious differences in the average ranking obtained by various improved algorithms.

The results of Friedman statistic F_{ID} on 10 test cases for various improved algorithms (HGWO, NIMASS, ADCSO, PSO, HLO-PSO, SLGA and HGA-VNS) are shown in Table 19. The calculated result of Friedman statistic F_{ID} is 3.953. Because k-1 = 6, (k-1)(n-1) = 54 and $\alpha = 0.05$, $F_{(6,54)}$ is distributed according to F distribution with degrees of freedom 2 and 18, the critical value of $F_{(6,54)}$ for $\alpha = 0.05$ is 2.272. Because $F_{ID} = 3.953 > 2.272$, the null hypothesis is rejected in $\alpha = 0.05$. Similarly, when $\alpha = 0.01$ and $F_{(6,54)}$ is distributed according to F distribution with degrees of freedom 6 and 54, the critical value of $F_{(6,54)}$ for $\alpha = 0.01$ is 3.156. Because $F_{ID} = 3.9531 > 3.156$, the null hypothesis is rejected in $\alpha = 0.01$. The results of Friedman test in $\alpha = 0.05$ and 0.01 show that there are obvious differences in the average ranking obtained by HGWO, NIMASS, ADCSO, PSO, HLO-PSO, SLGA and HGA-VNS.

Table 18
The results of Friedman statistic χ^2 in different significance level.

k	df	α	χ^2	$\chi^2_{\alpha(k-1)}$	H_0	H_1
7	6	0.05	18.311	12.592	×	√
7	6	0.01	18.311	16.812	×	√

From the analysis in Fig. 16 and Tables 16–19, the solution result of HGA-VNS is obviously better than that of other algorithms participating in the comparison, and there are significant differences in the performance of the three algorithms participating in the comparison. The performance of HGA-VNS is better than that of HGWO, NIMASS, ADCSO, PSO, HLO-PSO, and SLGA.

5. Cases study

Taking a machine shop of enterprise F as an example, the existing scheduling scheme of this shop is optimized to give the optimal

Table 19
The results of Friedman statistic F_{ID} in different significance level.

k	df	α	F_{ID}	$F_{[(k-1),\ (k-1)(n-1)]}$	H_0	H_1
7	6	0.05	3.953	2.272	×	√
7	6	0.01	3.953	3.156	×	√

scheduling scheme considering load balancing.

5.1. Case background

(1) Enterprise situation.

Enterprise F is a large state-owned enterprise with excellent technical strength. It has more than 5000 employees, including more than 1200 engineers and technicians and more than 400 senior technicians, and has the top R&D capability in the industry. The enterprise has nearly 1800 sets of production equipment, including more than 200 sets of advanced equipment and more than 100 sets of large imported equipment, and the overall level of equipment is in the leading position in China. In recent years, the enterprise actively responds to the national call, independent research and development, pioneering, and is moving towards the goal of becoming an international first-class enterprise in the industry.

(2) Workshop production.

This workshop is responsible for processing a certain type of part of one of the main products of enterprise F. There are 10 types of jobs for processing this type of part, and the number of processes for each type of workpiece varies, with a total of 51 operations, which is a typical FJSP. The workshop currently has 5 types of equipment that can process these jobs, with a total of 12 processing machines. Among them, the machining center can complete turning, milling and drilling, but not grinding, and the other machining machines can only complete 1 type of machining task. With the existing scheduling scheme of this shop for machining, it takes 181 min to produce one batch of this type of parts. The existing scheduling scheme is shown in Fig. 17, and the detailed information of the equipment in this shop is shown in Table 20.

There is a sequence between the processes of each job, each operation has its own optional processing machine, and each machine has its own corresponding processing time, and the detailed data is shown in Table 21.

5.2. Optimal scheduling solution

The HGA-VNS is applied to solve the FJSP of enterprise F. The main parameters of the algorithm are set as follows: the population size is 200, the crossover probability P_{cc} is 0.8, the mutation probability P_m is 0.3,

Table 20
Machine information.

No.	Equipment	No.	Equipment
1	Turning machines	7	Grinding machines
2	Turning machines	8	Grinding machines
3	Turning machines	9	Grinding machines
4	Milling machines	10	Drilling machines
5	Milling machines	11	Machining Center
6	Milling machines	12	Machining Center

the proportion of elite individuals P_e is 0.1, the selection probability of the combined crossover operator P_{cc} is 0.3, the selection probability of the combined mutation operator P_{mm} is 0.5, the maximum number of iterations is 100, the number of individuals selected in each tournament selection is 2, and the solution is performed with the objective function of minimizing the maximum completion time. The allocation of machines in the derived optimal scheduling scheme is shown in Table 22, and the Gantt chart of the obtained optimal scheduling scheme is shown in Fig. 18.

In Fig. 18, each colored rectangle represents a process for a type of workpiece, and the value in the rectangle represents the processing time for that process.

The makespan of the original scheduling scheme for producing this type of part is 181 min, and the makespan of the optimal scheduling scheme optimized with HGA-VNS is 154 min in Fig. 18. The makespan of the optimal scheduling scheme is reduced by 14.92 % compared with the original scheduling scheme, thus verifying the effectiveness of the proposed HGA-VNS.

6. Conclusion

To minimize the makespan and the machine work load balance in machining system, we have built the model of the flexible job shop scheduling (FJSP). Distinct from other literature, we consider both FJSP structures and objective features, and propose hybrid genetic algorithm with variable neighborhood search (HGA-VNS) as a means of addressing the mechanical production process.

In the HGA-VNS algorithm, each solution is represented by a chromosome consists of two parts, where the first part is the code of the machining machine number, and the second part chromosome is the code of the machining process number. Then, considering the slow convergence speed of the previous algorithms, a combined methods in

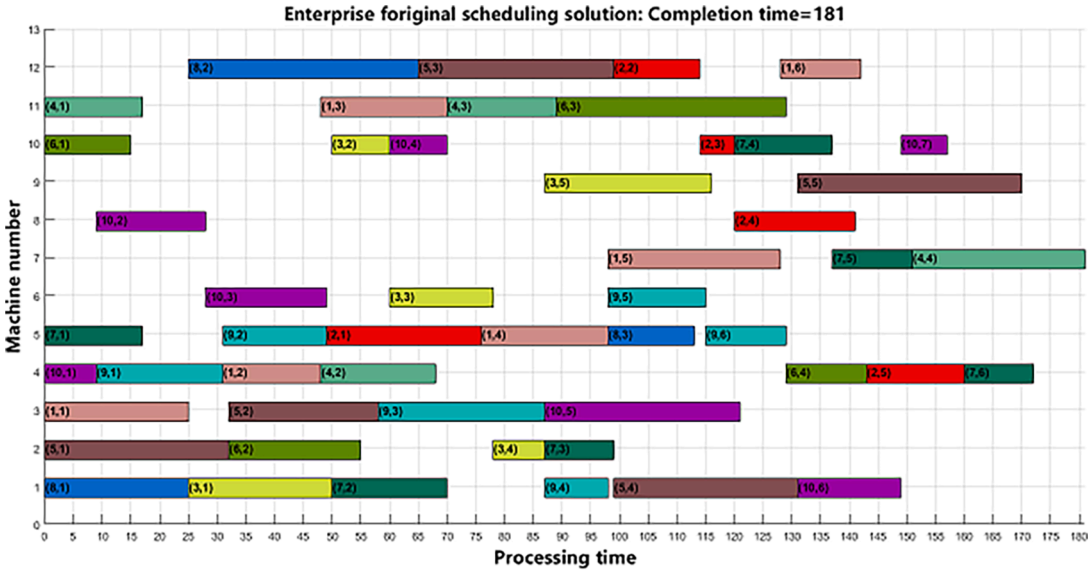


Fig. 17. Existing scheduling scheme.

Table 21
Processing machine and processing time.

Job number	Number of operations	Operation number	Process name	Optional processing machine code	Processing time
J_1	6	O_{11}	Turning	1,2,3,11,12	22,24,25,26,24
		O_{12}	Milling	4,5,6,11,12	17,22,16,23,19
		O_{13}	Turning	1,2,3,11,12	12,17,14,22,18
		O_{14}	Milling	4,5,6,11,12	21,22,19,22,21
		O_{15}	Grinding	7,8,9	30,32,32,
		O_{16}	Drilling	10,11,12	14,16,14
J_2	5	O_{21}	Milling	4,5,6,11,12	30,27,25,34,31
		O_{22}	Turning	1,2,3,11,12	15,12,14,16,15
		O_{23}	Drilling	10,11,12	6,10,8
		O_{24}	Grinding	7,8,9	20,21,23
		O_{25}	Milling	4,5,6,11,12	17,16,19,22,20
J_3	5	O_{31}	Turning	1,2,3,11,12	25,30,28,35,32
		O_{32}	Drilling	10,11,12	10,15,12
		O_{33}	Milling	4,5,6,11,12	22,27,18,30,29
		O_{34}	Turning	1,2,3,11,12	10,9,13,14,9
		O_{35}	Grinding	7,8,9	32,35,29
J_4	4	O_{41}	Turning	1,2,3,11,12	12,15,13,17,15
		O_{42}	Milling	4,5,6,11,12	20,22,18,25,24
		O_{43}	Turning	1,2,3,11,12	20,19,21,19,17
		O_{44}	Grinding	7,8,9	30,29,32
J_5	5	O_{51}	Turning	1,2,3,11,12	33,32,38,40,37
		O_{52}	Turning	1,2,3,11,12	36,28,26,35,32
		O_{53}	Turning	1,2,3,11,12	32,31,29,38,34
		O_{54}	Milling	4,5,6,11,12	32,35,29,35,33
		O_{55}	Grinding	7,8,9	41,34,39
J_6	4	O_{61}	Drilling	10,11,12	15,18,16
		O_{62}	Turning	1,2,3,11,12	22,23,25,36,35
		O_{63}	Turning	1,2,3,11,12	36,35,37,40,41
		O_{64}	Milling	4,5,6,11,12	14,15,17,17,14
J_7	6	O_{71}	Milling	4,5,6,11,12	18,17,19,22,21
		O_{72}	Turning	1,2,3,11,12	20,26,24,30,27
		O_{73}	Turning	1,2,3,11,12	14,12,15,16,14
		O_{74}	Drilling	10,11,12	17,20,19
		O_{75}	Grinding	7,8,9	14,18,16
		O_{76}	Milling	4,5,6,11,12	12,15,14,16,15
J_8	3	O_{81}	Turning	1,2,3,11,12	25,26,29,32,31
		O_{82}	Turning	1,2,3,11,12	40,42,44,45,40
		O_{83}	Milling	4,5,6,11,12	16,15,17,21,19
J_9	6	O_{91}	Milling	4,5,6,11,12	22,27,24,30,27
		O_{92}	Milling	4,5,6,11,12	25,18,22,25,24
		O_{93}	Turning	1,2,3,11,12	35,30,29,40,36
		O_{94}	Turning	1,2,3,11,12	11,15,14,19,14
		O_{95}	Milling	4,5,6,11,12	22,18,17,26,22
		O_{96}	Milling	4,5,6,11,12	16,14,19,20,17
J_{10}	7	O_{101}	Milling	4,5,6,11,12	9,10,11,14,10
		O_{102}	Grinding	7,8,9	22,19,21
		O_{103}	Milling	4,5,6,11,12	23,25,21,26,24
		O_{104}	Drilling	10,11,12	10,15,13
		O_{105}	Turning	1,2,3,11,12	36,40,34,45,42
		O_{106}	Turning	1,2,3,11,12	18,25,19,29,26
		O_{107}	Drilling	10,11,12	8,10,9

crossover and mutation operators that considers the machine work load balance is proposed. In addition, a local search approach that carry out for key processes on the critical path which reduces the number of invalid transformations is proposed.

Table 22
Processing machine and processing time.

Job number	Sequence of process	Machine assignment
J_1	O_{11}-O_{12}-O_{13}-O_{14}-O_{15}-O_{16}	M_{12}-M_6-M_1-M_5-M_7-M_{10}
J_2	O_{21}-O_{22}-O_{23}-O_{24}-O_{25}	M_5-M_2-M_{10}-M_9-M_4
J_3	O_{31}-O_{32}-O_{33}-O_{34}-O_{35}	M_{11}-M_{10}-M_6-M_2-M_9
J_4	O_{41}-O_{42}-O_{43}-O_{44}	M_1-M_6-M_{11}-M_8
J_5	O_{51}-O_{52}-O_{53}-O_{54}-O_{55}	M_2-M_3-M_3-M_{12}-M_8
J_6	O_{61}-O_{62}-O_{63}-O_{64}	M_{10}-M_1-M_2-M_4
J_7	O_{71}-O_{72}-O_{73}-O_{74}-O_{75}-O_{76}	M_5-M_1-M_{12}-M_{10}-M_7-M_4
J_8	O_{81}-O_{82}-O_{83}	M_2-M_1-M_5
J_9	O_{91}-O_{92}-O_{93}-O_{94}-O_{95}-O_{96}	M_4-M_5-M_{11}-M_1-M_6-M_5
J_{10}	O_{101}-O_{102}-O_{103}-O_{104}-O_{105}-O_{106}-O_{107}	M_4-M_8-M_6-M_{10}-M_3-M_1-M_{10}

In order to investigate the most suitable parameter combination of HGA-VNS, a four-factor three-level orthogonal experiment scheme is designed. Although we could not obtain the theoretically optimal combination of parameters for the algorithm, we obtained the best combination of parameters in the design experiment. Subsequently, we design two sets of experiments to test and analyze the performance of the proposed HGA-VNS. The test results show that, compared with other algorithms, the average value and optimal value obtained by HGA-VNS are significantly better. To better validate the robustness of our proposed algorithm, we need more and different types of test sets. In addition, the proposed HGA-VNS is applied to optimize practical FJSP in enterprise F. The results show the makespan of the optimal scheduling scheme is reduced by 14.92 % compared with the original scheduling scheme, and HGA-VNS can obtain more efficient and economic solutions.

In our proposed HGA-VNS, in addition to the advantages mentioned above, there are also some limitations. In the investigation of the parameters of the algorithm, we have only explored the effect of crossover probability and variance probability on the performance of the algorithm, and have not yet investigated the effect of population size on the performance of the algorithm. In addition, there is a lack of corresponding investigation in our study regarding the proportion of elite individuals retained in the selection operator in Section 3.5.

In the future, there is still a lot of work to be done by us, where the problem modeling should be as close to the actual production situation as possible. Future works are as follows: (1) to consider more objective functions and develop new multi-objective algorithms for meeting the designer's requirements to solving more realistic green intelligent optimization problems, such as the use of low-carbon equipment and green building consumables, etc.; (2) to combine our own experience in front-line production and take into account the influencing factors of scheduling in actual production, so that can solve large-scale scheduling problems in line with actual production.

CRediT authorship contribution statement

Kexin Sun: Conceptualization, Data curation, Formal analysis, Methodology, Validation, Visualization, Writing – original draft, Writing – review & editing. **Debin Zheng:** Conceptualization, Data curation, Formal analysis, Methodology, Validation, Visualization, Writing – original draft, Writing – review & editing. **Haohao Song:** Conceptualization, Data curation, Formal analysis, Methodology, Validation, Visualization, Writing – original draft, Writing – review & editing. **Zhiwen Cheng:** Data curation, Formal analysis, Project administration, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Xudong Lang:** Investigation, Methodology, Resources. **Weidong Yuan:** Investigation, Methodology, Resources. **Jiquan Wang:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Resources.

Declaration of Competing Interest

The authors declare that they have no known competing financial

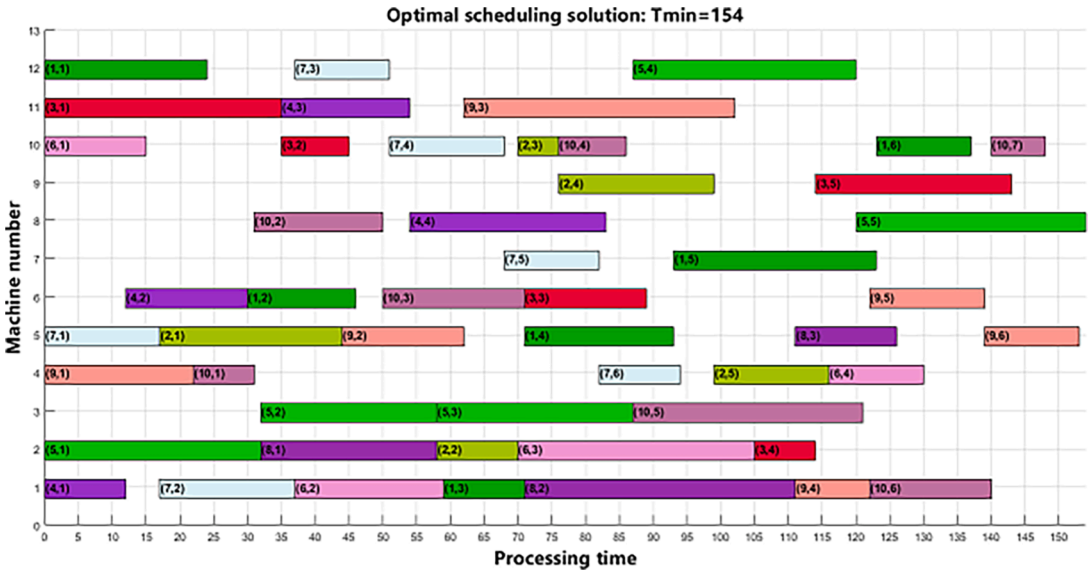


Fig. 18. HGA-VNS solves the optimal scheduling scheme for FJSP.

interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by National Social Science Foundation of China (Grant No. 21BGL174). The authors thank the anonymous reviewers for their valuable and constructive comments that greatly helped improve the quality and completeness of the paper.

References

Al-Hinai, N., & ElMekkawy, T. Y. (2011). An efficient hybridized genetic algorithm architecture for the flexible job shop scheduling problem. *Flexible Services and Manufacturing Journal, 23*, 64–85. https://doi.org/10.1007/s10696-010-9067-y

Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research, 41*(3), 157–183. https://doi.org/10.1007/BF02023073

Brucker, P., & Schlie, R. (1990). Job-shop scheduling with multi-purpose machines. *Computing (Vienna/New York), 45*(4), 369–375.

Buddala, R., & Mahapatra, S. S. (2019). An integrated approach for scheduling flexible job-shop using teaching-learning-based optimization method. *Journal of Industrial Engineering International, 15*(1), 181–192. https://doi.org/10.1007/s40092-018-0280-8

Chang, H.-C., Chen, Y.-P., Liu, T.-K., & Chou, J.-H. (2015). Solving the flexible job shop scheduling problem with makespan optimization by using a hybrid taguchi-genetic algorithm. *IEEE Access, 3*, 1740–1754. https://doi.org/10.1109/ACCESS.2015.2481463

Chang, H.-C., & Liu, T.-K. (2017). Optimisation of distributed manufacturing flexible job shop scheduling by using hybrid genetic algorithms. *Journal of Intelligent Manufacturing, 28*(8), 1973–1986. https://doi.org/10.1007/s10845-015-1084-y

Chen, H., Ihlow, J., & Lehmann, C. (1999), 10-15 May 1999. *A genetic algorithm for flexible job-shop scheduling.* Paper presented at the Proceedings 1999 IEEE International Conference on Robotics and Automation (Cat. No.99CH36288C).

Chen, R., Yang, B., Li, S., & Wang, S. (2020). A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. *Computers & Industrial Engineering, 149*. https://doi.org/10.1016/j.cie.2020.106778

Cheng, R. W., Gen, M., & Tsujimura, Y. (1999). A tutorial survey of job-shop scheduling problems using genetic algorithms, part II: Hybrid genetic search strategies. *Computers & Industrial Engineering, 36*(2), 343–364. https://doi.org/10.1016/s0360-8352(99)00136-9

Derrac, J., Garcia, S., Molina, D., & Herrera, F. (2011). A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation, 1*(1), 3–18. https://doi.org/10.1016/j.swevo.2011.02.002

Ding, H., & Gu, X. (2020a). Hybrid of human learning optimization algorithm and particle swarm optimization algorithm with scheduling strategies for the flexible job-shop scheduling problem. *Neurocomputing, 414*, 313–332. https://doi.org/10.1016/j.neucom.2020.07.004

Ding, H., & Gu, X. (2020b). Improved particle swarm optimization algorithm based novel encoding and decoding schemes for flexible job shop scheduling problem. *Computers & Operations Research, 121*. https://doi.org/10.1016/j.cor.2020.104951

Driss, I., Mouss, K. N., & Laggoun, A. (2015). A new genetic algorithm for flexible job-shop scheduling problems. *Journal of Mechanical Science and Technology, 29*(3), 1273–1281. https://doi.org/10.1007/s12206-015-0242-7

Fan, J., Zhang, C., Liu, Q., Shen, W., & Gao, Q. (2022). An improved genetic algorithm for flexible job shop scheduling problem considering reconfigurable machine tools with limited auxiliary modules. *Journal of Manufacturing Systems, 62*, 650–667. https://doi.org/10.1016/j.jmsy.2022.01.014

Fekih, A., Hadda, H., Kacem, I., & Hadj-Alouane, A. B. (2021). A hybrid genetic Tabu search algorithm for minimising total completion time in a flexible job-shop scheduling problem. *European Journal of Industrial Engineering, 14*(6), 763–781. https://doi.org/10.1504/EJIE.2020.112479

Gao, J., Gen, M., Sun, L., & Zhao, X. (2007). A hybrid of genetic algorithm and bottleneck shifting for multiobjective flexible job shop scheduling problems. *Computers & Industrial Engineering, 53*(1), 149–162. https://doi.org/10.1016/j.cie.2007.04.010

Garey, M. R., Johnson, D. S., & Sethi, R. J. M.o.o.r (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of operations research, 1*(2), 117–129.

Goldberg, D. E., & Deb, K. (1991). A Comparative Analysis of Selection Schemes Used in Genetic Algorithms. In G. J. E. Rawlins (Ed.), *Foundations of Genetic Algorithms* (Vol. 1, pp. 69–93). Elsevier.

Gu, X., Huang, M., & Liang, X. (2019). An improved genetic algorithm with adaptive variable neighborhood search for FJSP. *Algorithms, 12*(11). https://doi.org/10.3390/a12110243

Iman, R. L., Davenport, J. M., & Methods. (1980). Approximations of the critical region of the fbietkan statistic. *Communications in Statistics-Theory Methods, 9*(6), 571–595.

Ishikawa, S., Kubota, R., & Horio, K. (2015). Effective hierarchical optimization by a hierarchical multi-space competitive genetic algorithm for the flexible job-shop scheduling problem. *Expert Systems with Applications, 42*(24), 9434–9440. https://doi.org/10.1016/j.eswa.2015.08.003

Johnson, D., Chen, G., & Lu, Y.-Q. (2022). Multi-agent reinforcement learning for real-time dynamic production scheduling in a robot assembly cell. *IEEE Robotics and Automation Letters, 7*(3), 7684–7691. https://doi.org/10.1109/LRA.2022.3184795

Jamrus, T., Chien, C.-F., Gen, M., & Sethanan, K. (2018). Hybrid particle swarm optimization combined with genetic operators for flexible job-shop scheduling under uncertain processing time for semiconductor manufacturing. *IEEE Transactions on Semiconductor Manufacturing, 31*(1), 32–41. https://doi.org/10.1109/TSM.2017.2758380

Jiang, T.-H. (2018). Flexible job shop scheduling problem with hybrid grey wolf optimization algorithm. *Control and Decision, 33*(3), 503–508. https://doi.org/10.13195/j.kzyjc.2017.0124

Jiang, T.-H., & Zhang, C. (2019). Adaptive discrete cat swarm optimisation algorithm for the flexible job shop problem. *International Journal of Bio-Inspired Computation, 13*(3), 199–208. https://doi.org/10.1504/ijbic.2019.10020503

Karimi, H., Rahmati, S. H. A., & Zandieh, M. (2012). An efficient knowledge-based algorithm for the flexible job shop scheduling problem. *Knowledge-Based Systems, 36*, 236-244. doi:10.1016/ j.knosys. 2012.04.001.

Lim, K. C. W., Wong, L.-P., & Chin, J.-F. (2022). Simulated-annealing-based hyper-heuristic for flexible job-shop scheduling. *Engineering Optimization Online.*. https://doi.org/10.1080/0305215X.2022.2106477

Meziane, M.-E.-A., & Taghezout, N. (2018). A hybrid genetic algorithm with a neighborhood function for flexible job shop scheduling. *Multiagent and Grid Systems, 14*(2), 161–175. https://doi.org/10.3233/mgs-180286

Mladenović, N., & Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research, 24*(11), 1097–1100. https://doi.org/10.1016/S0305-0548(97)00031-2

Nouiri, M., Bekrar, A., Jemai, A., Niar, S., & Ammari, A. C. (2018). An effective and distributed particle swarm optimization algorithm for flexible job-shop scheduling problem. *Journal of Intelligent Manufacturing, 29*(3), 603–615. https://doi.org/10.1007/s10845-015-1039-3

Serna, N. J. E., Seck-Tuoh-Mora, J. C., Medina-Marin, J., Hernandez-Romero, N., & Armenta, J. R. C. (2021). A global-local neighborhood search algorithm and tabu search for flexible job shop scheduling problem. *PeerJ Computer Science, 7*(2), e574.

Pan, Z., Lei, D., & Wang, L. (2022). A bi-population evolutionary algorithm with feedback for energy-efficient fuzzy flexible job shop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics: Systems, 52*(8), 5295–5307. https://doi.org/10.1109/TSMC.2021.3120702

Pezzella, F., Morganti, G., & Ciaschetti, G. (2008). A genetic algorithm for the flexible job-shop scheduling problem. *Computers and Operations Research, 35*(10), 3202–3212. https://doi.org/10.1016/j.cor.2007.02.014

Qais, M. H., Hasanien, H. M., & Alghuwainem, S. (2018). Augmented grey wolf optimizer for grid-connected PMSG-based wind energy conversion systems. *Applied Soft Computing Journal, 69*, 504–515. https://doi.org/10.1016/j.asoc.2018.05.006

Qin, Q., Cheng, S., Zhang, Q., Li, L., & Shi, Y. (2016). Particle swarm optimization with interswarm interactive learning strategy. *IEEE Trans Cybern, 46*(10), 2238–2251. https://doi.org/10.1109/TCYB.2015.2474153

Tang, J., Zhang, G., Lin, B., & Zhang, B. (2011). A Hybrid Algorithm for Flexible Job-shop Scheduling Problem. In C. Ran & G. Yang (Eds.), *Ceis 2011* (Vol. 15).

Wang, C., Li, Y., & Li, X. (2021). Solving flexible job shop scheduling problem by a multi-swarm collaborative genetic algorithm. *Journal of Systems Engineering and Electronics, 32*(2), 261–271. https://doi.org/10.23919/jsee.2021.000023

Wang, Y. L., & Zhu, Q. W. (2021). A hybrid genetic algorithm for flexible job shop scheduling problem with sequence-dependent setup times and job lag times. *IEEE access., 9*, 104864–104873. https://doi.org/10.1109/ACCESS.2021.3096007

Wang, Y. M., Yin, H. L., & Qin, K. D. (2013). A novel genetic algorithm for flexible job shop scheduling problems with machine disruptions. *International Journal of Advanced Manufacturing Technology, 68*(5–8), 1317–1326. https://doi.org/10.1007/s00170-013-4923-z

Xiong, W., & Fu, D. (2018). A new immune multi-agent system for the flexible job shop scheduling problem. *Journal of Intelligent Manufacturing, 29*(4), 857–873. https://doi.org/10.1007/s10845-015-1137-2

Yan, J., Liu, Z., Zhang, C., Zhang, T., Zhang, Y., & Yang, C. (2021). Research on flexible job shop scheduling under finite transportation conditions for digital twin workshop. *Robotics and Computer-Integrated Manufacturing., 72*. https://doi.org/10.1016/j.rcim.2021.102198

Yang, J., & Soh, C. K. (1997). Structural optimization by genetic algorithms with tournament selection. *Journal of Computing in Civil Engineering, 11*(3), 195–200. https://doi.org/10.1061/(ASCE)0887-3801(1997)11:3(195)

Yazdani, M., Amiri, M., & Zandieh, M. (2010). Flexible job-shop scheduling with parallel variable neighborhood search algorithm. *Expert Systems with Applications, 37*(1), 678–687. https://doi.org/10.1016/j.eswa.2009.06.007

Yuan, Y., Xu, H., & Yang, J. (2013). A hybrid harmony search algorithm for the flexible job shop scheduling problem. *Applied Soft Computing Journal, 13*(7), 3259–3272. https://doi.org/10.1016/j.asoc.2013.02.013

Zhang, C., Rao, Y., & Li, P. (2008). An effective hybrid genetic algorithm for the job shop scheduling problem. *International Journal of Advanced Manufacturing Technology, 39*(9–10), 965–974. https://doi.org/10.1007/s00170-007-1354-8

Zhang, G., Gao, L., & Shi, Y. (2011). An effective genetic algorithm for the flexible job-shop scheduling problem. *Expert Systems with Applications, 38*(4), 3563–3573. https://doi.org/10.1016/j.eswa.2010.08.145

Zhang, G., Zhang, L., Song, X., Wang, Y., & Zhou, C. (2019). A variable neighborhood search based genetic algorithm for flexible job shop scheduling problem. *Cluster Computing-The Journal of Networks Software Tools and Applications., 22*(5), 11561–11572. https://doi.org/10.1007/s10586-017-1420-4

Zarrouk, R., Bennour, I. E., & Jemai, A. (2019). A two-level particle swarm optimization algorithm for the flexible job shop scheduling problem. *Swarm Intelligence, 13*(2), 145–168. https://doi.org/10.1007/s11721-019-00167-w

Zhang, X., & Ming, X. (2020). Reference subsystems for Smart Manufacturing Collaborative System (SMCS) from multi-processes, multi-intersections and multi-operators. *Enterprise Information Systems, 14*(3), 282–307. https://doi.org/10.1080/17517575.2019.1694705